NORTHWESTERN UNIVERSITY

Optimizing pulse for gate and decoherence for superconducting qubits

A DISSERTATION

SUBMITTED TO THE GRADUATE SCHOOL
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

for the degree

DOCTOR OF PHILOSOPHY

Department of Physics and Astronomy

By

Yunwei Lu

EVANSTON, ILLINOIS

June 2026

ABSTRACT

Superconducting cavity–transmon architectures provide a powerful route to quantum information processing by combining long-lived cavity memories with the controllability of nonlinear ancillae. Their performance, however, is limited by two consequences of the same coupling mechanism: high-fidelity cavity control requires accurate dynamics in large Hilbert spaces, while ancilla noise is inherited by the cavity and undermines the bias structure needed for bosonic error correction.

This thesis develops practical methods to address both limitations. In the control setting, I first refvisit GRAPE with hard-coded gradients, eliminating the memory bottleneck of automatic differentiation while preserving runtime scaling and enabling optimal control in large cavity–transmon Hilbert spaces. I then use this framework to optimize ancilla pulses in an echoed conditional displacement protocol. these pulses suppress crosstalk errors by orders of magnitude and produce Bell-cat states with substantially lower infidelity than standard DRAG-based control.

In the noise setting, I show that cavity dephasing induced by 1/f flux noise in a flux-tunable transmon can be strongly suppressed by a weak off-resonant drive that creates driven flux-insensitive points. At these operating points, the drive-induced energy shift opposes the Lamb-shift fluctuations to first order, rendering the cavity transition frequency insensitive to transmon frequency noise. Monte Carlo simulations using realistic parameters predict an approximately twentyfold enhancement of cavity dephasing time, and the protocol extends naturally to bosonic encodings: a single drive simultaneously suppresses flux-noise sensitivity across all relevant transitions of a binomial code and reduces dephasing in both rails of a dual-rail qubit.

Taken together, these results provide a unified strategy for improving both operation fidelity and memory protection in cavity–transmon quantum processors, using hardware-efficient protocols

compatible with existing experimental platforms.

ACKNOWLEDGEMENTS

A young scamp about fifteen years old, Isidore Duval by name, and called, for convenience, Zidore, took care of this pensioner, gave him his measure of oats and fodder in winter, and in summer was supposed to change his pasturing place four times a day, so that he might have plenty of fresh grass.

The animal, almost crippled, lifted with difficulty his legs, large at the knees and swollen above the hoofs. His coat, which was no longer curried, looked like white hair, and his long eyelashes gave to his eyes a sad expression.

When Zidore took the animal to pasture, he had to pull on the rope with all his might, because it walked so slowly.

LIST OF PUBLICATIONS

Placeholder: Add your peer-reviewed publications here (published, accepted, and submitted), formatted per your program requirements.

DEDICATION

Placeholder dedication text.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

CHAPTER 1
INTRODUCTION

Quantum computing matters because it addresses computational tasks for which classical methods appear fundamentally inadequate. Feynman famously argued that the state space of a generic quantum many-body system grows exponentially with system size, so that classical computers cannot efficiently simulate quantum physics in full generality, whereas a controllable quantum device should be able to do so more naturally [1, 2]. This prospect makes quantum computing especially compelling for quantum simulation, with anticipated applications ranging from molecular electronic structure, ground-state energies, and reaction dynamics relevant to chemistry and drug design [3, 4], to strongly correlated materials, high-temperature superconductivity, and lattice gauge theories in physics [5, 6], and to emerging problems in computational biology and medicine such as protein folding, photosynthetic energy-transfer models, and quantum-assisted drug discovery [7, 8]. Beyond simulation, quantum algorithms can also provide rigorous advantages for certain classical problems. Shor's algorithm yields an exponential speedup for integer factorization and discrete logarithms, thereby threatening widely used public-key cryptosystems and motivating the ongoing transition to post-quantum cryptography [9–11], while Grover's algorithm offers a quadratic query speedup for unstructured search [12, 13]. Recent surveys further identify finance as an important application area for fault-tolerant quantum algorithms, with proposed uses in portfolio optimization, Monte Carlo derivative pricing, and risk analysis [10, 14, 15]. Other directions, including quantum machine learning and quantum optimization, further broaden the field's scope, although their practical advantage remains an active research question [16, 17]. Together, these scientific and computational prospects have driven strong worldwide efforts, as well as substantial

public and private investment, to build practical quantum computing platforms [18, 19].

A practical quantum computer must be realized in a physical system of effective two-level degrees of freedom, or qubits, that can be initialized, coherently controlled, entangled, and measured with high fidelity, while maintaining coherence for times long compared with the duration of gate operations [20, 21]. Several hardware platforms satisfy these requirements to different degrees. Nitrogen-vacancy centers in diamond offer exceptionally long coherence times even at room temperature, but scalable fabrication and controlled interconnects remain challenging [22, 23]. Trapped ions provide some of the best coherence times and gate fidelities demonstrated so far, although gate operations are relatively slow and large-scale integration remains technically demanding [24, 25]. Neutral-atom processors have demonstrated large qubit arrays together with dynamically reconfigurable connectivity, making them attractive for both programmable quantum simulation and gate-based computation [26, 27]. Photonic platforms are especially natural for quantum communication and networking, but deterministic two-qubit gates remain difficult because photons interact only weakly in ordinary optical media [28, 29]. Superconducting circuits, by contrast, combine fast gate operations with lithographically scalable fabrication and mature microwave control techniques [30, 31]. No single platform has yet simultaneously optimized all of the requirements for large-scale quantum computation, and current hardware development is therefore shaped by trade-offs among coherence, connectivity, control speed, and manufacturability [20, 21]. Among gate-based implementations, superconducting circuits are currently one of the most mature platforms, as reflected both in major public roadmaps from IBM and Google and in recent large-scale error-correction demonstrations on superconducting processors [31–34]. This thesis therefore focuses exclusively on superconducting circuits.

Superconducting qubits encode quantum information in quantized macroscopic variables of Josephson circuits, where nonlinear circuit elements allow selected levels of an electrical mode to

be isolated and controlled as a qubit [30, 31]. Over the past two decades, a wide variety of designs has emerged, each optimizing a different balance between anharmonicity, tunability, coherence, and noise protection. The charge qubit, or Cooper-pair box, provided the first clear demonstration of coherent control in a superconducting qubit, but its strong sensitivity to offset-charge fluctuations limited its coherence [35, 36]. The flux qubit instead encodes information in superpositions of opposite persistent-current states, offering strong anharmonicity and convenient magnetic control, although flux noise becomes a central decoherence channel [37, 38]. The transmon, obtained by shunting the Cooper-pair box with a large capacitor, exponentially suppresses charge noise at the cost of reduced anharmonicity [39, 40]. Fluxonium and especially heavy fluxonium recover large anharmonicity while supporting very long coherence through the use of a superinductor [41, 42]. Related protected designs, including the bifluxon and the 0-π qubit, seek hardware-level suppression of dominant noise channels by engineering symmetry and disjoint wave-function support into the circuit itself [43, 44]. More exotic proposals, such as current-mirror and $\cos 2\phi$-based protected circuits, extend this strategy further [45, 46]. Despite this zoology, the transmon has become the workhorse of both academia and industry because it combines conceptual simplicity, reliable fabrication, high-fidelity control, and fast microwave readout in a scalable architecture [31, 47]. In present-day devices, this platform supports coherence times from tens to hundreds of microseconds together with high-fidelity gate operations, and it has been scaled from 105-qubit-class processors such as Google's Willow chip to IBM's 1,121-qubit Condor processor [34, 48]. This thesis therefore focuses on superconducting circuits, with particular emphasis on the transmon architecture.

Alongside transmon-based processors, superconducting microwave cavities provide a distinct and complementary paradigm for quantum information processing. In both three-dimensional and planar implementations, a cavity is a linear superconducting resonator that can store microwave photons with coherence times far exceeding those of typical transmon qubits, while uni-

versal control is usually supplied by coupling the cavity to a nonlinear ancilla such as a transmon [49, 50]. Early progress in three-dimensional circuit QED established the exceptional coherence of machined cavities [51, 52], and continued improvements in materials, fabrication, and surface treatment pushed 3D cavity photon lifetimes from the millisecond regime to tens of milliseconds and ultimately beyond $2\,\mathrm{s}$ in state-of-the-art niobium resonators at millikelvin temperatures [53–55]. Planar two-dimensional superconducting resonators generally suffer stronger dielectric and surface-interface losses and therefore do not yet match the best 3D cavities [56, 57]; nevertheless, recent on-chip quantum-memory designs have exceeded the millisecond scale, demonstrating that 2D architectures can also reach remarkably long coherence when carefully engineered [58, 59]. The underlying physical advantage of both 2D and 3D cavities is that, as nearly ideal linear oscillators, they avoid the dominant charge, flux, and critical-current noise channels associated with Josephson qubits, so photon loss is typically the primary error mechanism while pure dephasing remains comparatively weak [60, 61]. This strong bias toward photon-loss errors, together with the large Hilbert space of a harmonic oscillator, makes superconducting cavities especially attractive for bosonic encoding and quantum error correction.

A superconducting cavity by itself is only a harmonic oscillator, with equally spaced energy levels, so linear drives cannot selectively address individual transitions or generate the non-Gaussian operations required for universal quantum control [62, 63]. In circuit QED, the standard solution is to couple the cavity dispersively to an anharmonic ancilla, typically a transmon, so that the qubit and cavity frequencies acquire state-dependent shifts characterized by the dispersive coupling χ [62, 64]. This interaction enables a rich toolbox for bosonic control. In the strong-dispersive regime, the selective number-dependent arbitrary phase (SNAP) gate applies independently programmable phases to chosen Fock states by driving ancilla transitions at frequencies $\omega_q + n\chi$; together with cavity displacements, SNAP gates provide universal single-cavity control

[65, 66]. Their reliance on spectrally selective ancilla pulses, however, makes them increasingly vulnerable to Stark shifts and spectator-mode crosstalk in multimode devices [63, 67]. Echoed conditional displacement (ECD) gates address the same problem differently: by combining conditional cavity displacements with ancilla echo pulses, they realize fast universal control even when the bare dispersive coupling is weak, and because the ancilla operation is not photon-number selective they are generally less sensitive than SNAP to crosstalk, although not completely immune to it [67, 68]. More recently, conditional-NOT displacement (CNOD) has extended this philosophy to multimode architectures, showing that a single ancilla can efficiently control and entangle several weakly coupled oscillators [50, 69]. In parallel, optimized SNAP-displacement sequences and improved cavity-control protocols have further reduced coherent errors and expanded the accessible family of bosonic states [70, 71]. For two-cavity operations, however, one generally needs an activated bilinear coupling between modes. Early demonstrations realized programmable beam-splitter interactions through a driven transmon ancilla [72, 73], whereas newer architectures employ dedicated parametric couplers such as SNAIL-based three-wave mixers and parity-protected converters to obtain fast swaps, high on/off ratios, and low idle-time backaction on the memories [74, 75]. More recently, linear quantum couplers such as the LINC have been proposed and realized to push this idea further by remaining effectively linear when idle while activating cleaner parametric interactions under drive [76, 77]. Together, these developments define the modern control toolbox for bosonic circuit QED and have enabled beam-splitter-based logical operations, including dual-rail cavity encodings and error-detectable entangling gates between bosonic qubits [78, 79].

Superconducting cavities are especially attractive for bosonic quantum error correction because their large Hilbert space and strongly biased noise channels allow a single oscillator mode to store a logical qubit with hardware-efficient redundancy [50, 61]. The central idea is to encode logical

information in nonclassical superpositions of Fock states or phase-space states chosen so that the dominant physical error—typically single-photon loss—maps the system into a distinguishable error manifold that can be detected and corrected before the logical information is destroyed [50, 61]. Among the major bosonic code families, cat codes encode information in superpositions of coherent states $|\pm\alpha\rangle$, for which single-photon loss changes the photon-number parity and appears as a logical bit-flip-type error [80, 81]. Binomial codes instead use carefully engineered finite superpositions of Fock states and can be designed to correct photon loss, photon gain, and dephasing; the lowest-order example is $|0_L\rangle = (|0\rangle + |4\rangle)/\sqrt{2}$ and $|1_L\rangle = |2\rangle$ [82, 83]. Gottesman–Kitaev–Preskill (GKP) codes encode a logical qubit in grid states of oscillator phase space and are tailored to correct small displacement errors in both quadratures [84, 85]. A complementary dual-rail encoding stores the logical qubit nonlocally across two cavities, with $|0_L\rangle = |01\rangle$ and $|1_L\rangle = |10\rangle$; in this case a photon-loss event maps the state outside the code space as an erasure-type error that is comparatively easy to detect [50, 78]. Experimentally, bosonic QEC in superconducting cavities has progressed from the first break-even cat-code experiment [61, 86], through the first full GKP error-correction demonstrations in circuit QED [85, 87], to beyond-break-even real-time correction with GKP states [85, 88] and with binomial codes [61, 89]. Cat-code experiments have likewise demonstrated extended logical lifetimes and exponentially suppressed bit-flip rates as the phase-space separation is increased [90, 91]. This progress relies critically on noise bias: bosonic encodings are most powerful when photon loss remains the dominant cavity error, because then increasing the encoding size can strongly reduce logical failure rates, whereas comparable dephasing substantially weakens that advantage [83, 92]. For that reason, suppressing cavity dephasing—even at the cost of additional hardware or control complexity—remains central to the bosonic-QEC program [61, 92].

As cavity-based processors move toward larger bosonic encodings and multimode architec-

tures, high-fidelity control must be designed in an increasingly large effective Hilbert space. Even a single cavity mode is infinite-dimensional and must therefore be truncated at sufficiently high photon number for accurate simulation, while the inclusion of additional cavity modes causes the retained Hilbert-space dimension to grow multiplicatively and hence very rapidly with system size [63, 67]. A second source of complexity appears when strong microwave drives are used to accelerate cavity control: the transmon ancilla can no longer be treated as an ideal two-level system, because large amplitudes may populate high-lying excited states and eventually induce transmon ionization into weakly bound or unconfined states [93, 94]. In this regime, closed-form pulse design becomes increasingly unreliable, since leakage, spectator-mode Stark shifts, and drive-induced nonlinearities all contribute coherently to the dynamics [67, 94]. Achieving high-fidelity control therefore generally requires numerical quantum optimal control, in which the full truncated Hamiltonian and experimental constraints are incorporated explicitly to optimize pulse shapes while suppressing leakage and crosstalk [95, 96].

One of the prominent techniques for implementing quantum optimal control is Gradient Ascent Pulse Engineering (GRAPE). The central goal of optimal control is to adjust control parameters in such a way that the infidelity of the desired state transfer or gate operation is minimized. In GRAPE, the minimization is based on gradient descent and thus generally requires the evaluation of gradients. Automatic differentiation (AD) [97] is a convenient tool that has also made an impact on quantum optimal control with GRAPE [98–101]. Internally, AD builds a computational graph and applies the chain rule to automatically compute the gradient of a given function.

However, the convenience of AD comes at the cost of memory required for storing the full computational graph. The use of semi-automatic differentiation (semi-AD) [102] partially reduces the memory overhead compared to full automatic differentiation (full-AD). This reduction can still be insufficient to tackle optimal control for quantum systems of rapidly growing size [103, 104].

This motivates one of my works revisit of the original GRAPE scheme [105], which is based on hard-coded gradients (HG) and has the smallest possible memory cost.

1.1 Thesis outline

This thesis is organized as follows. Chapter 2 introduces the GRAPE framework and derives hard-coded analytical gradients for key cost contributions used in state-transfer and gate-optimization problems. The chapter also discusses the numerical evaluation of these gradients using the scaling-and-squaring method and the auxiliary matrix method, culminating in a memory-efficient forward-backward propagation scheme.

Chapter 3 presents a scaling analysis of runtime and memory usage for hard-coded gradients, full-AD, and semi-AD. Benchmark studies for state-transfer and gate-optimization problems illustrate the scaling behavior. A decision tree is developed to guide the choice of the optimal numerical strategy.

Chapter 4 turns to an application of GRAPE: the design of detuning-robust control pulses for transmon gates. The optimization procedure is detailed, and the resulting pulses are benchmarked against conventional DRAG pulses and pulses from Q-CTRL.

Chapter 5 summarizes the key findings and discusses opportunities for future research.

Appendix A discusses the scaling of the number of matrix-vector multiplications μ with Hilbert space dimension for several concrete Hamiltonians. Appendix B presents an alternative hard-coded gradient scheme based on matrix diagonalization and its scaling analysis.

CHAPTER 2

GRAPE BASED ON HARD-CODED GRADIENTS

This chapter introduces the Gradient Ascent Pulse Engineering (GRAPE) framework and derives analytical gradient expressions for key cost contributions. The numerical evaluation of these expressions—via the scaling-and-squaring method, the auxiliary matrix method, and a forward-backward propagation scheme—is then discussed in detail. The results of this chapter form the foundation for the scaling analysis and benchmarking presented in Chapter 3.

2.1 Brief sketch of GRAPE

I briefly sketch the basics of GRAPE [105], mainly to establish the notation used in subsequent sections. Consider a system described by the Hamiltonian

$$H(t) = H_s + a(t)\, h_c. \tag{2.1}$$

Here, H_s denotes the static system Hamiltonian and h_c is an operator coupling the classical control field $a(t)$ to the system.[1]Quantum optimal control aims to adjust $a(t)$ such that a desired unitary gate or state transfer is performed within a given time interval $0 \leq t \leq T$. To facilitate optimization, the total control time T is divided into N intervals of duration $\Delta t = T/N$, and the control field is specified by the amplitudes at the discrete times $t_n = n\Delta t$ ($n = 1, 2, \ldots, N$). The discretization interval Δt is chosen such that control amplitudes are approximately constant within each Δt. This treatment is close to modeling the actual drive output of an arbitrary waveform gen-

[1]For simplicity, I consider one control channel with real-valued $a(t)$. Generalizations to multiple control channels and complex a are straightforward.

erator, where Δt can be chosen to align with the available resolution. I denote the time-discretized values by

$$a_n \coloneqq a(t_n), \tag{2.2}$$

and group these to form the vector $\mathbf{a} \in \mathbb{R}^N$. Under the influence of this piecewise-constant control field, the system's quantum state $|\psi_n\rangle \coloneqq |\psi(t_n)\rangle$ evolves by a single time step according to

$$|\psi_n\rangle = U_n |\psi_{n-1}\rangle. \tag{2.3}$$

Here, U_n is the short-time propagator

$$U_n \coloneqq U(t_n, t_{n-1}) = e^{-iH_n \Delta t} \qquad (\hbar = 1), \tag{2.4}$$

where $H_n = H_s + a_n h_c$. Optimization of the control field proceeds via minimization of a cost function C. Typically, C includes the state-transfer or gate infidelity, along with additional cost contributions employed to moderate pulse characteristics such as maximal power or bandwidth. To this end a composite cost function is constructed, $C(\mathbf{a}) = \sum_\nu \alpha_\nu C_\nu(\mathbf{a})$, where α_ν are empirically chosen weight factors and C_ν are individual cost contributions. GRAPE utilizes the method of steepest descent to minimize $C(\mathbf{a})$ by updating the control amplitudes according to the cost function gradient:

$$\mathbf{a}^{(j+1)} = \mathbf{a}^{(j)} - \eta_j \nabla_{\mathbf{a}} C(\mathbf{a}^{(j)}). \tag{2.5}$$

Here, j enumerates steepest-descent iterations and $\eta_j > 0$ is the learning rate governing the step size.

2.2 Analytical gradients

A critical ingredient for GRAPE is the computation of cost function gradients. One popular option is to leverage automatic differentiation for this purpose, such as implemented in `Tensorflow` [106] and `Autograd` [107]. A clear advantage of this strategy is the ease by which complicated cost function gradients are evaluated automatically at runtime. This convenience, however, is bought at the cost of significant memory consumption which can be prohibitive for large quantum systems. Therefore, I am motivated to revisit hard-coded gradients in a computationally efficient scheme for key cost contributions.

I discuss the analytical gradients of two key cost contributions and provide detailed expressions for the gradients of other cost contributions.

2.2.1 State-transfer infidelity

In state-transfer problems, a central goal of quantum optimal control is to minimize the state-transfer infidelity given by

$$C_1^{st} = 1 - |\langle\phi_T|\psi_N\rangle|^2 \,, \tag{2.6}$$

where $|\phi_T\rangle$ is the target state and $|\psi_N\rangle$ is the state realized at final time t_N. The gradient of C_1^{st} takes the form

$$\frac{\partial C_1^{st}}{\partial a_n} = -\langle\phi_T|U_N\cdots\frac{\partial U_n}{\partial a_n}\cdots U_1|\psi_0\rangle\langle\psi_N|\phi_T\rangle - \mathrm{c.\,c.} = -\langle\phi_n|\frac{\partial U_n}{\partial a_n}|\psi_{n-1}\rangle\langle\psi_N|\phi_T\rangle - \mathrm{c.\,c.}, \tag{2.7}$$

where the state $|\phi_n\rangle$ obeys the recursive relation

$$|\phi_n\rangle = U_{n+1}^{\dagger}|\phi_{n+1}\rangle. \tag{2.8}$$

This can be interpreted as backward propagation.

2.2.2 Gate infidelity

In addition to state transfer, unitary gates are another operation of interest. Gate optimization can be achieved by minimizing the gate infidelity $C_1^g = 1 - \left|\operatorname{tr}\left(U_T^{\dagger}U_R\right)/d\right|^2$. Here, U_T is the target unitary gate and $U_R = U_N \cdots U_1$ is the actually realized gate. For a given orthonormal basis $\{|\psi_0^h\rangle\}_h$ (h enumerates the basis states), the gradient of C_1^g can be written as

$$\frac{\partial C_1^g}{\partial a_n} = -\frac{1}{d^2}\sum_h \langle\phi_n^h|\frac{\partial U_n}{\partial a_n}|\psi_{n-1}^h\rangle \sum_{h'}\langle\psi_N^{h'}|\phi_N^{h'}\rangle - \text{c. c.}, \tag{2.9}$$

where $|\psi_n^h\rangle = U_n \cdots U_1|\psi_0^h\rangle$. The state $|\phi_n^h\rangle$ is obtained by applying the target unitary U_T to basis state $\{|\psi_0^h\rangle\}_h$ and backward propagating the result to time t_n,

$$|\phi_n^h\rangle = U_{n+1}^{\dagger}\cdots U_N^{\dagger}U_T|\psi_0^h\rangle. \tag{2.10}$$

2.2.3 Additional cost contributions

The cost contribution

$$C_2^{st} = \frac{1}{N}\sum_{n'=1}^{N}\langle\psi_{n'}|\Omega|\psi_{n'}\rangle \tag{2.11}$$

penalizes the integrated expectation value of the Hermitian operator Ω. The gradient of this cost contribution can be written as

$$\begin{aligned}\frac{\partial C_2^{st}}{\partial a_n} &= \frac{1}{N}\sum_{n'=1}^{N}\langle\psi_{n'}|\Omega\frac{\partial}{\partial a_n}\prod_{n''=1}^{n'}U_{n''}|\psi_0\rangle + c.c. \\ &= \frac{1}{N}\Big(\sum_{n'=n}^{N}\prod_{n''=n+1}^{n'}\langle\psi_{n'}|\Omega U_{n''}\frac{\partial}{\partial a_n}U_n|\psi_{n-1}\rangle\Big) + c.c. \\ &= \frac{2}{N}\,\mathrm{Re}(\langle\Phi_n|\frac{\partial}{\partial a_n}U_n|\psi_{n-1}\rangle), \end{aligned} \tag{2.12}$$

with

$$|\Phi_n\rangle := \sum_{n'=n}^{N} U_{n+1}^{\dagger}\cdots U_{n'}^{\dagger}\Omega|\psi_{n'}\rangle \tag{2.13}$$

The vector $|\Phi_n\rangle$ obeys the recursion relation

$$|\Phi_n\rangle = U_{n+1}^{\dagger}|\Phi_{n+1}\rangle + \Omega|\psi_n\rangle. \tag{2.14}$$

Due to this relation, the expression in Eq. (2.12) can be evaluated with a forward-backward scheme analogous to the one discussed in Sec. 2.3.3.

The cost contribution $C_3^{st} = 1 - \frac{1}{N}\sum_{n'=1}^{N}\left|\langle\phi_T \mid \psi_{n'}\rangle\right|^2$ sums up infidelities from all time steps (as opposed to recording the final state infidelity only, as in C_1^{st}). This steers the system towards its target state more rapidly, thus helping to reduce the duration of state transfer. Similar to Eq. (2.12), the gradient of C_3^{st} is:

$$\frac{\partial C_3^{st}}{\partial a_n} = -\frac{2}{N}\,\mathrm{Re}\left(\langle\Phi_{T,n}|\frac{\partial}{\partial a_n}U_n|\psi_{n-1}\rangle\right), \tag{2.15}$$

with

$$|\Phi_{T,n}\rangle := \sum_{n'=n}^{N} U_{n+1}^{\dagger}\cdots U_{n'}^{\dagger}|\phi_T\rangle\langle\phi_T|\psi_{n'}\rangle. \tag{2.16}$$

The vector obeys the recursive relation

$$|\Phi_{T,n}\rangle = U_{n+1}^{\dagger}|\Phi_{T,n+1}\rangle + |\phi_T\rangle\langle\phi_T|\psi_n\rangle. \tag{2.17}$$

This allows the use of a forward-backward scheme to evaluate the gradient expression in Eq. (2.15).

For gate operations, the cost contribution analogous to C_3^{st} is given by

$$C_3^g = 1 - \sum_{n'=1}^{N} \left| \operatorname{tr}\left(U_T^{\dagger} U_{n',1}\right)\right|^2 / Nd^2, \tag{2.18}$$

where $U_{n',1} = U_{n'} \cdots U_1$ describes the evolution from time step 1 to n'. The gradient is

$$\frac{\partial C_3^g}{\partial a_n} = -\frac{2}{Nd^2} \sum_{h=1}^{d} \operatorname{Re}\left(\langle \Phi_{T,n}^h | \frac{\partial}{\partial a_n} U_n | \psi_{n-1}^h \rangle\right), \tag{2.19}$$

with

$$|\Phi_{T,n}^h\rangle := \sum_{n'=n}^{N} U_{n+1}^{\dagger} \cdots U_{n'}^{\dagger} U_T |\psi_0^h\rangle \operatorname{tr}(U_{n',1} U_T^{\dagger}). \tag{2.20}$$

Here, $\{|\psi_0^h\rangle\}$ forms an arbitrary orthonormal basis and $h = 1, 2, \ldots, d$ enumerates the basis states. The vector $|\Phi_{T,n}^h\rangle$ obeys the recursion relation

$$|\Phi_{T,n}^h\rangle = U_{n+1}^{\dagger}|\Phi_{T,n+1}^h\rangle + U_T|\psi_0^h\rangle \operatorname{tr}(U_{n,1} U_T^{\dagger}), \tag{2.21}$$

again enabling the evaluation of the gradient expression with a forward-backward propagation scheme.

Equations related to the gradient of cost functions such as (2.7) and (2.9) provide the analytical expressions needed for gradient descent. In the next two sections, I discuss how to numerically evaluate these expressions in a memory-efficient way.

2.3 Numerical implementation of hard-coded gradients

The calculation of gradients in equations including (2.7) and (2.9) requires numerical evaluation of expressions of the form $U_n|\Psi\rangle$ and $\frac{\partial U_n}{\partial a_n}|\Psi\rangle$. Here, the short-time propagator U_n is given by a matrix exponential e^A and $A = -iH_n\Delta t$.

Numerically evaluating $e^A|\Psi\rangle$ is not without challenge. As pointed out by Moler and Van Loan [108], several methods exist, yet none of them is truly optimal. Three methods ranked highly in Ref. [108] are based on solving ordinary differential equations, matrix diagonalization, and scaling and squaring (combined with series expansion). ODE solving, in this context, tends to be most expensive in runtime [102]. I focus mainly on scaling and squaring but comment on situations where matrix diagonalization is preferable in Sec. 3.3.

2.3.1 State propagation evaluation by scaling and squaring

Scaling and squaring [109] is based on the identity

$$e^A = \left(e^{A/s}\right)^s, \tag{2.22}$$

where the right-hand side matrix exponential now involves the matrix $B = A/s$ which has a norm that is reduced compared to $||A||$. This generally allows for a lower truncation order m of the exponential series[2],

$$e^B \approx e_m^B := \sum_{q=0}^{m} \frac{B^q}{q!}. \tag{2.23}$$

[2]While Chebyshev expansion has faster convergence than Taylor expansion, their numerical implementations have the same runtime scaling. Here, I focus on Taylor expansion due to its simplicity.

Typically, m is smaller than the one required for the original e^A. The goal is to approximate the state vector $e^A|\Psi\rangle \approx (e_m^B)^s|\Psi\rangle$ where

$$e_m^B|\Psi\rangle = \left(1 + B + \frac{B^2}{2!} + \cdots + \frac{B^m}{m!}\right)|\Psi\rangle = |\Psi\rangle + B|\Psi\rangle + \frac{B\left(B|\Psi\rangle\right)}{2} + \cdots + \frac{B\left(\cdots\left(B|\Psi\rangle\right)\right)}{m!}\,. \tag{2.24}$$

According to the last expression, approximation of $e^A|\Psi\rangle$ can thus proceed by repeated application of $B = A/s$ to a vector, without ever invoking matrix-matrix multiplication. This boosts runtime performance from $O(d^3)$ to $O(d^2)$.

To proceed with the evaluation of $e^A|\Psi\rangle$, the truncation order m and scaling factor s are determined in such a way that the error remains below the error tolerance τ,

$$\varepsilon_{A,|\Psi\rangle}(m,s) := \frac{\left|\left|e^A|\Psi\rangle - [\![(e_m^B)^s|\Psi\rangle]\!]\right|\right|}{||e^A|\Psi\rangle||} < \tau. \tag{2.25}$$

Here, $[\![\bullet]\!]$ denotes the floating point representation of $\bullet$. Since this error $\varepsilon_{A,|\Psi\rangle}$ is difficult and costly to evaluate, I instead derive an upper bound $E_A(m,s)$ and resort to the inequality

$$\varepsilon_{A,|\Psi\rangle}(m,s) < E_A(m,s) \leq \tau. \tag{2.26}$$

2.3.1 Determining the upper bound $E_A(m,s)$

The approximation of $e^A|\Psi\rangle$ required the determination of an upper bound $E_A(m,s)$ for the error $\varepsilon_{A,|\Psi\rangle}(m,s)$ [Eq. (2.25)]. Here, I show how to acquire such bound.

Considering the expression for the error [Eq. (2.25)], the denominator is simply 1. The error

$\varepsilon_{A,|\Psi\rangle}$ is bounded as follows:

$$\begin{aligned}\varepsilon_{A,|\Psi\rangle} &= \left|\left|e^A|\Psi\rangle - (e_m^B)^s|\Psi\rangle + (e_m^B)^s|\Psi\rangle - [\![(e_m^B)^s|\Psi\rangle]\!]\right|\right|_2 \\ &\leq \underbrace{\left|\left|e^A|\Psi\rangle - (e_m^B)^s|\Psi\rangle\right|\right|_2}_{\varepsilon_t} + \underbrace{\left|\left|(e_m^B)^s|\Psi\rangle - [\![(e_m^B)^s|\Psi\rangle]\!]\right|\right|_2}_{\varepsilon_r}.\end{aligned} \tag{2.27}$$

The error ε_t arises from the truncation of the Taylor series and rounding error ε_r is due to finite-precision arithmetics. I proceed to derive the upper bounds for these two errors separately.

Upper bound for the truncation error ε_t. The truncation error can be rewritten as

$$\varepsilon_t = \left|\left|e^A|\Psi\rangle - (e^B - R_m(B))^s|\Psi\rangle\right|\right|_2 = \left|\left|\sum_{i=1}^{s} \frac{s!}{i!(s-i)!}(-R_m(B))^i (e^B)^{s-i}|\Psi\rangle\right|\right|_2, \tag{2.28}$$

where $R_m(B) = \sum_{q=m+1}^{\infty} B^q/q!$ denotes the matrix-valued error of the truncated exponential series. Employing the triangle inequality and submultiplicativity [110], one finds the bound

$$\varepsilon_t \leq \sum_{i=1}^{s} \frac{s!}{i!(s-i)!}\left(R_m(||B||_2)\right)^i \left|\left|e^B\right|\right|_2^{s-i} |||\Psi\rangle||_2 = \sum_{i=1}^{s} \frac{s!}{i!(s-i)!}\left(R_m(||B||_2)\right)^i. \tag{2.29}$$

Here the second equality utilizes the fact that the state is normalized and e^B is unitary.

This upper bound must be evaluated numerically to determine m and s. However, computing the matrix 2-norm is inconvenient since it requires the use of an eigensolver. To mitigate this, I

switch to the matrix 1-norm by exploiting[3]

$$||B||_2 \leq ||B||_1 . \tag{2.30}$$

Monotonicity of R_m leads to

$$\varepsilon_t \leq \sum_{i=1}^{s} \frac{s!}{i!(s-i)!} \big(R_m(||B||_1)\big)^i < sR_m(||B||_1) \frac{1-\big(sR_m(||B||_1)\big)^s}{1-sR_m(||B||_1)}. \tag{2.31}$$

Upper bound for the rounding error ε_r. The rounding error ε_r originates from the finite precision of the floating-point representation of real numbers as well as the finite accuracy of basic arithmetic operations. The floating-point representation $[\![\xi]\!]$ for the real number ξ can be written as $[\![\xi]\!] = \xi(1+\delta)$ where $\delta = \delta(\xi)$ is the relative deviation [110]. Its magnitude is always smaller than machine precision, i.e. $|\delta| < u$. Similarly, the complex number z obeys

$$[\![z]\!] = z(1+\delta) \tag{2.32}$$

with $|\delta| \leq u$. Arithmetic operations such as addition and multiplication with two complex numbers z_1, z_2 incur a relative deviation $\delta = \delta(z_1, z_2)$ such that

$$[\![z_1 + z_2]\!] = ([\![z_1]\!] + [\![z_2]\!])(1+\delta),\ |\delta| \leq u, \quad [\![z_1 z_2]\!] = [\![z_1]\!][\![z_2]\!](1+\delta),\ |\delta| \leq u', \tag{2.33}$$

where $u' := 2\sqrt{2}u/(1-2u)$ [110]. The relative deviation shown in Eqs. (2.32), (2.33) generally depend on both numbers involved as well as the operation in question. In the following, different δ are distinguished with subscript ν.

[3]Generally, $||B||_2 \leq \sqrt{||B||_\infty ||B||_1}$ [111] holds true for any B. When B is anti-Hermitian, $||B||_\infty = ||B||_1$ leads to the inequality in Eq. (2.30).

Using Eqs. (2.32), (2.33), I derive an upper bound for the rounding error in the evaluation of the dot product, which is the basic arithmetic operation in evaluating $e_m^B|\Psi\rangle$ according to Eq. (2.24).

Upper bound for the rounding error associated with evaluating the dot product. The rounding error incurred when evaluating the dot product is given by $|\Sigma_d - [\![\Sigma_d]\!]|$ with $\Sigma_d = \boldsymbol{w}^{\mathrm{T}}\boldsymbol{z}$ ($\boldsymbol{w}, \boldsymbol{z} \in \mathbb{C}^d$). Here, $\boldsymbol{w}^{\mathrm{T}}$ is a row of $B = i\delta t H/s$. Based on Eqs. (2.32) and (2.33), one has

$$|[\![\Sigma_1]\!] - \Sigma_1| = |[\![w_1 z_1]\!] - w_1 z_1| < |w_1||z_1|\Big|\prod_{\nu=1}^{3}(1+\delta_\nu) - 1\Big| < \gamma_3 |w_1||z_1|, \tag{2.34}$$

where $\left|\prod_{\nu=1}^{n}(1+\delta_\nu) - 1\right| < \frac{nu'}{1-nu'}$ [110], and $\gamma_n := \frac{nu'}{1-nu'}$. For $d = 2$, the upper bound of the error is

$$|[\![\Sigma_2]\!] - \Sigma_2| = |([\![\Sigma_1]\!] + [\![w_2 z_2]\!])(1+\delta_1) - \Sigma_2| < \gamma_4 \sum_{i=1}^{2} |w_i||z_i|. \tag{2.35}$$

The general upper bound for arbitrary d is:

$$|[\![\Sigma_d]\!] - \Sigma_d| < \gamma_{d+2} \sum_{j=1}^{d} |w_j||z_j|. \tag{2.36}$$

Since $\boldsymbol{w}^{\mathrm{T}}$ is proportional to a row of the Hamiltonian matrix, sparsity of H implies sparsity of $\boldsymbol{w}^{\mathrm{T}}$. Vanishing entries in $\boldsymbol{w}^{\mathrm{T}}$ are special because they do not incur rounding errors. Thus, one can obtain a stronger upper bound

$$|[\![\Sigma_d]\!] - \Sigma_d| < \gamma_{\sigma+2} \sum_{j=1}^{d} |w_j||z_j| \tag{2.37}$$

where σ is the number of non-zero elements in $\boldsymbol{w}^{\mathrm{T}}$.

Upper bound for the rounding error of $e_m^B|\Psi\rangle$. The vector $e_m^B|\Psi\rangle$ is evaluated recursively via

$$b_j = \frac{B}{j} b_{j-1}, \qquad e_j^B b_0 = b_{j-1} + b_j \tag{2.38}$$

where $j = 1, 2, \cdots, m$ and b_0 is a vector representation of $|\Psi\rangle$ under a choice of orthonormal basis. Before evaluating the upper bound for the error $\left|\left|[\![e_m^B b_0]\!] - e_m^B b_0\right|\right|_2$, I first derive an upper bound for the error of a vector component $\left|[\![e_m^B b_0]\!]^{(i)} - (e_m^B b_0)^{(i)}\right|$.

For the case $m = 0$:

$$\left|[\![e_0^B b_0]\!]^{(i)} - (e_0^B b_0)^{(i)}\right| = \left|[\![b_0]\!]^{(i)} - b_0^{(i)}\right| < \gamma_2 |b_0^{(i)}|. \tag{2.39}$$

For $m = 1$:

$$\begin{aligned} |[\![e_1^B b_0]\!]^{(i)} - (e_1^B b_0)^{(i)}| &= \left|\left([\![b_0]\!]^{(i)} + [\![b_1]\!]^{(i)}\right)(1+\delta_1) - (e_1^B b_0)^{(i)}\right| \qquad (2.40) \\ &= \left|[\![b_0]\!]^{(i)}(1+\delta_1) + \frac{[\![Ab_0]\!]^{(i)}}{s} \prod_{\nu=1}^{3}(1+\delta_\nu)^3 - (e_1^B b_0)^{(i)}\right| \overset{(2.37)}{<} \gamma_3 |b_0^{(i)}| + \gamma_{\sigma_i+5} \sum_l |B^{(il)}||b_0^{(l)}|, \end{aligned}$$

where σ_i is the number of non-zero elements in the ith row of B. The general upper bound for arbitrary m is:

$$\left|[\![e_m^B b_0]\!]^{(i)} - (e_m^B b_0)^{(i)}\right| < \sum_{k=0}^{m} \gamma_{k(\sigma_i+2)+m+2} \left(\frac{|B|^k}{k!}|b_0|\right)^i, \tag{2.41}$$

where $|B|, |b_0|$ denote the matrix and the vector with elements $|B^{il}|$, $|b_0^i|$, respectively. Defining

$$\sigma' := \max_i \sigma_i \tag{2.42}$$

and using Eq. (2.41):

$$\begin{aligned}\left|\left|[\![e_m^B b_0]\!]-(e_m^B b_0)\right|\right|_2 &= \left|\left|\left|[\![e_m^B b_0]\!]-(e_m^B b_0)\right|\right|\right|_2 < \sum_{k=0}^{m}\gamma_{k(\sigma'+2)+m+2}\left|\left|\frac{|B|^k}{k!}|b_0|\right|\right|_2 \\ &< \sum_{k=0}^{m}\gamma_{k(\sigma'+2)+m+2}\frac{\left|\left|\left||B|\right|\right|\right|_2^k}{k!}\left|\left|\left||b_0|\right|\right|\right|_2 \overset{(2.30)}{<} \sum_{k=0}^{m}\gamma_{k(\sigma'+2)+m+2}\frac{\left|\left|\left||B|\right|\right|\right|_1^k}{k!}\left|\left|\left||b_0|\right|\right|\right|_2 = \beta||b_0||_2 \end{aligned} \tag{2.43}$$

with $\beta := \sum_{k=0}^{m}\gamma_{k(\sigma'+2)+m+2}\frac{||B||_1^k}{k!}$. Here, $\beta||b_0||_2$ is the desired upper bound for the rounding error of $e_m^B|\Psi\rangle$.

Upper bound for the error ε_r. The vector $(e_m^B)^s b_0$ is evaluated by the recursive relation

$$c_l = e_m^B c_{l-1}, \qquad l = 1, 2, \cdots, s, \tag{2.44}$$

where $c_0 = b_0$. I illustrate how to bound the error $\varepsilon_r = \left|\left|[\![(e_m^B)^s b_0]\!]-(e_m^B)^s b_0\right|\right|_2$ for $s = 1, 2$, and give the general result subsequently.

For $s = 1$:

$$\left|\left|[\![e_m^B b_0]\!]-e_m^B b_0\right|\right|_2 \overset{(2.43)}{<} \beta||b_0||_2 = \beta. \tag{2.45}$$

For $s = 2$:

$$\begin{aligned}\left|\left|[\![(e_m^B)^2 b_0]\!]-(e_m^B)^2 b_0\right|\right|_2 &\le \left|\left|[\![(e_m^B)^2 b_0]\!]-e_m^B[\![e_m^B b_0]\!]\right|\right|_2 + \left|\left|e_m^B[\![e_m^B b_0]\!]-(e_m^B)^2 b_0\right|\right|_2 \\ &< \left|\left|[\![e_m^B[\![e_m^B b_0]\!]]\!]-e_m^B[\![e_m^B b_0]\!]\right|\right|_2 + \beta\left|\left|e_m^B\right|\right|_2, \end{aligned} \tag{2.46}$$

where the last step is enabled by Eq. (2.45). The matrix norm in the second term is bounded by

$$\left|\left|e_m^B\right|\right|_2 = \left|\left|e^B - R_m(B)\right|\right|_2 \le 1 + R_m(||B||_2) \overset{(2.30)}{\le} 1 + R_m(||B||_1) =: \alpha. \tag{2.47}$$

Since Eq. (2.43) holds for any complex vector b_0, the error in Eq. (2.46) is further bounded by:

$$\left|\left|[\![(e_m^B)^2 b_0]\!] - (e_m^B)^2 b_0\right|\right|_2 \overset{(2.43)}{<} \beta \left|\left|[\![e_m^B b_0]\!]\right|\right|_2 + \beta\alpha = \beta \left|\left|[\![e_m^B b_0]\!] - e_m^B b_0 + e_m^B b_0\right|\right|_2 + \beta\alpha \tag{2.48}$$

$$\overset{(2.45)}{<} \beta(\beta + \alpha) + \beta\alpha. \tag{2.49}$$

Generalizing this strategy further yields the expression for the upper bound of ε_r:

$$\begin{aligned}\varepsilon_r = \left|\left|[\![(e_m^B)^s b_0]\!] - (e_m^B)^s b_0\right|\right|_2 &\leq \sum_{l=0}^{s-1} \left|\left|(e_m^B)^l [\![(e_m^B)^{s-l} b_0]\!] - (e_m^B)^{l+1} [\![(e_m^B)^{s-l-1} b_0]\!]\right|\right|_2 \\ &< \beta \sum_{l=0}^{s-1} (\alpha+\beta)^{s-l-1}\alpha^l = (\alpha+\beta)^s - \alpha^s. \end{aligned} \tag{2.50}$$

Combining the upper bounds for ε_r and ε_t, I return to Eq. (2.27) and obtain

$$\varepsilon_{A,|\Psi\rangle} < E_A(m,s) = sR_m(||B||_1)\frac{1 - \left(sR_m(||B||_1)\right)^s}{1 - sR_m(||B||_1)} + (\alpha+\beta)^s - \alpha^s, \tag{2.51}$$

where $||B||_1 = ||A||_1 / s$, $\alpha = \alpha(||A||_1, m, s)$ and $\beta = \beta(\sigma', ||A||_1, m, s)$.

2.3.1 Choosing scaling factor and truncation order

For a given A when computing state propagation by scaling and squaring discussed in Sec. 2.3.1, the inequality (2.26) generally has multiple solutions for m and s. As shown in Sec. 2.3.1.1, the evaluation requires ms matrix-vector multiplications, which are observed to dominate the runtime. Since systematic minimization of ms requires significant runtime itself, I instead: (1) select solutions to Eq. (2.26) with the smallest s, which is denoted by $\mathbb{s}$, and (2) among these pairs, choose the one with smallest m (denoted by $\mathbb{m}$).

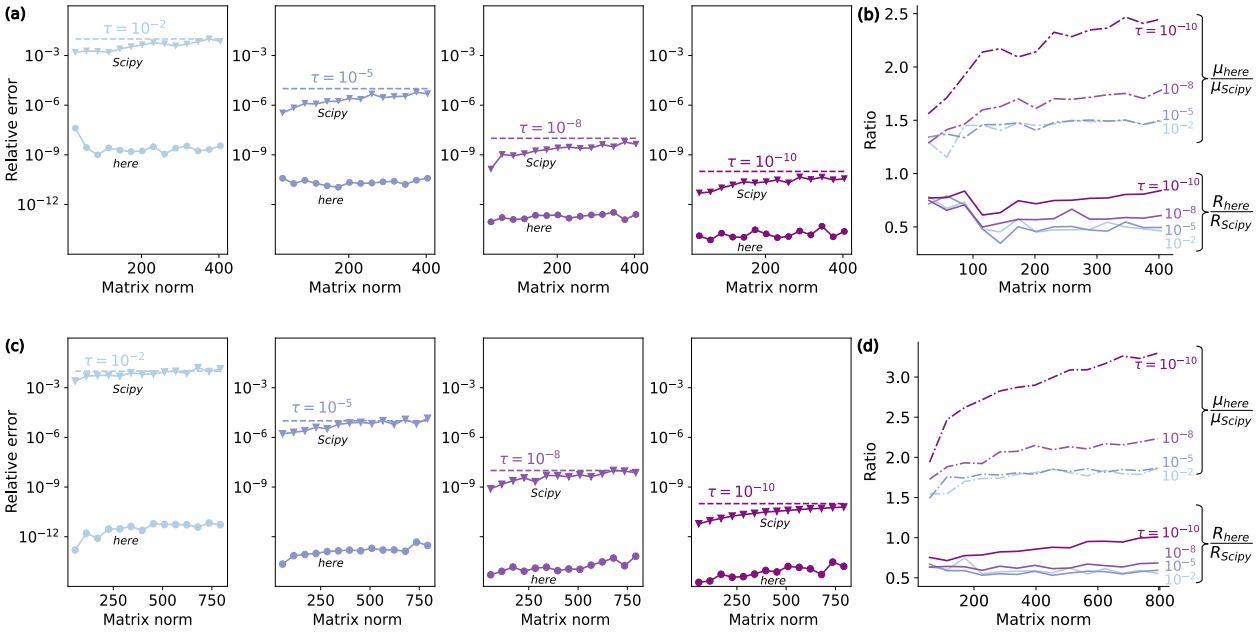

Figure 2.1: Comparison between scaling and squaring as implemented here and `Scipy`'s `expm_multiply`, considering relative error and runtime. (a), (c) Relative error versus matrix norm. (b), (d) Ratios of runtime and μ (i.e. number of matrix-vector multiplications) between the two implementations as functions of the matrix norm. The matrix norm $||-iH\Delta t||_2$ is varied by increasing $\Delta t \in [1, 10]$. In all panels, colors encode different error tolerances τ. The results of the first and second rows are obtained based on the cavity–transmon and three-coupled-transmons Hamiltonians, respectively. The system parameters of these two Hamiltonians are the same as those given in the captions of Figs. 3.1 and 3.2.

Thus, calculating $e^A|\Psi\rangle$ requires

$$\mu := \mathbb{ms} \tag{2.52}$$

matrix-vector multiplications. As shown in the following, for two concrete examples, this method for evaluating $e^A|\Psi\rangle$ can achieve better runtime performance and accuracy than `Scipy`'s `expm_multiply` function [112].

2.3.1 Numerical comparison between scaling and squaring implementations

Scaling and squaring is used to evaluate the propagator-state product, $e^{-iH\Delta t}|\Psi\rangle$. I compare relative error and runtime between the implementation of scaling and squaring presented here and `Scipy`'s implementation, `expm_multiply` [112]. The relative errors follow the general expression in Eq. (2.25), but differ in the specific choice of m and s. As a proxy for the exact propagator-state product, I apply Taylor expansion to the matrix exponential with 400-digit precision by `Sympy`; the result of the product converges at 30 digits after the decimal point. The runtime is measured with the help of the `Temci` package [113].

As the basis for this comparison, I use the examples of Hamiltonians (3.1) and (3.2) and implement sparse storage for them. The relative errors are shown in Fig. 2.1(a) and (c) as a function of the matrix norm $||-iH\Delta t||_2$, respectively. In both examples, the relative errors for the implementation presented here are bounded by the error tolerance τ, while the relative errors for `Scipy`'s implementation tend to exceed the tolerance for large norms. There are two reasons for this tendency. First, rounding errors are not considered in the derivation of the upper bound for the errors [112]; second, the truncation of e_m^B [Eq. (2.23)] is merely based on a heuristic criterion.

Even though the relative errors of `Scipy`'s implementation are worse, it achieves a smaller number of matrix-vector multiplications μ. This is illustrated in Fig. 2.1(b) and (d): the ratio of μ between the two implementations is always larger than 1. Although `Scipy`'s implementation spends less time on matrix-vector multiplications, the total runtime of `Scipy`'s implementation is larger, see Fig. 2.1(b) and (d). This is due to the larger runtime overhead of `Scipy`'s implementation for determining (m, s).

In conclusion, the implementation presented here has better numerical stability and runtime than `Scipy`'s implementation in these numerical experiments.

2.3.2 Computing the propagator derivative acting on a state: auxiliary matrix method

Having addressed the challenges associated with numerically evaluating $e^A|\Psi\rangle$, I now turn to the calculation of $\frac{\partial U_n}{\partial a_n}|\Psi\rangle$, which is crucial for computing cost function gradients as in Eqs. (2.7) and (2.9). This evaluation is based on the approximation

$$\frac{\partial U_n}{\partial a_n}|\Psi\rangle \approx \frac{\partial (e_m^{B_n})^s}{\partial a_n}|\Psi\rangle, \tag{2.53}$$

where $B_n = -iH_n\Delta t/s$. I proceed by using the auxiliary matrix method based on the identity [114, 115]

$$(e_m^{\mathcal{A}_n/s})^s \begin{pmatrix} 0 \\ |\Psi\rangle \end{pmatrix} = \begin{pmatrix} \frac{\partial (e_m^{B_n})^s}{\partial a_n}|\Psi\rangle \\ (e_m^{B_n})^s|\Psi\rangle \end{pmatrix}. \tag{2.54}$$

Here, $\mathcal{A}_n$ is defined as

$$\mathcal{A}_n := \begin{pmatrix} -iH_n\Delta t & -ih_c\Delta t \\ 0 & -iH_n\Delta t \end{pmatrix}, \tag{2.55}$$

which is a 2×2 matrix of operators each acting on the Hilbert space with dimension d. Once a basis is chosen, $\mathcal{A}_n$ can be represented as a $2d \times 2d$ matrix of complex numbers. Numerical evaluation of the left-hand side of the identity (2.54) gives a $2d$-dimensional vector. The first d entries of the vector correspond to $\partial (e_m^{B_n})^s/\partial a_n|\Psi\rangle$, which is an approximation of $\frac{\partial U_n}{\partial a_n}|\Psi\rangle$.

2.3.3 Efficient evaluation of hard-coded gradients

The analytical gradient expressions, such as ∇C_1^{st} in Eq. (2.7), are generally complicated and involve a large number of contributing quantities. Depending on grouping and ordering of operations, these quantities may need to be stored simultaneously or computed repeatedly with critical implications for runtime and memory usage. The scheme originally proposed by Khaneja *et al.* [105] strikes a favorable compromise that avoids the proliferation of stored intermediate state

vectors. More specifically, the final quantum state $|\psi_N\rangle$ is obtained by forward propagating $|\psi_0\rangle$. Then, backward propagation of $|\psi_N\rangle$ and $|\phi_T\rangle$ is carried out to calculate the matrix element $\langle\phi_n|\frac{\partial U_n}{\partial a_n}|\psi_{n-1}\rangle$ needed for the n-th component of the gradient, thus building up the gradient in descending order. For both propagation directions, only the current intermediate states are stored, leading to a memory cost of $\Theta(1)$.[4]

The forward-backward propagation scheme is illustrated in Fig. 2.2. This chapter has established the analytical and numerical machinery needed for evaluating cost function gradients in GRAPE without automatic differentiation. In the next chapter, I present a scaling analysis comparing the memory usage and runtime of hard-coded gradients with automatic differentiation approaches.

[4]Here, $f(x) = \Theta(g(x))$ is the usual Bachmann–Landau notation for $f(x)$ bounded above and below by $g(x)$ in the asymptotic sense.

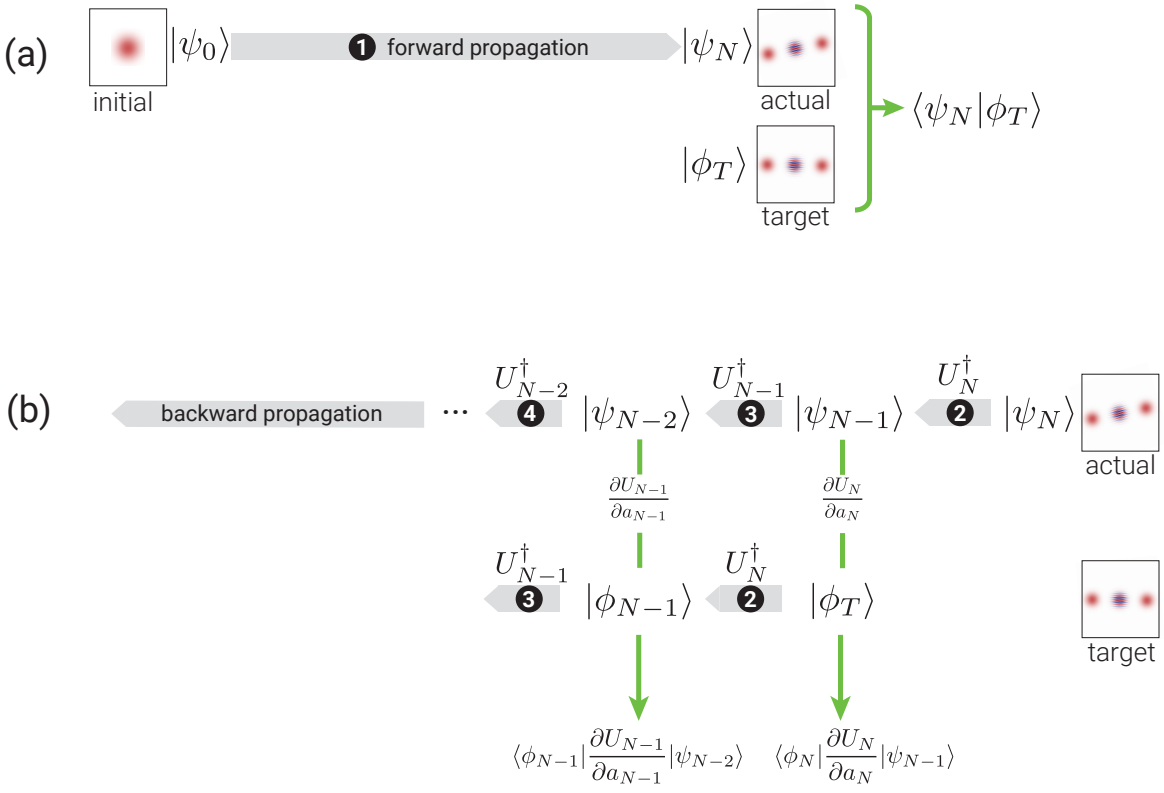

Figure 2.2: The forward-backward propagation scheme to compute the components of the gradient in Eq. (2.7). (a) Forward propagate the initial state by $U_N \cdots U_1$ to obtain the realized state $|\psi_N\rangle$ and calculate the overlap $\langle\psi_N|\phi_T\rangle$, where $|\phi_T\rangle$ is the target state. (b) Backward propagate $|\psi_N\rangle$ and $|\phi_T\rangle$ by the propagators $U_N^\dagger, U_{N-1}^\dagger, \cdots, U_1^\dagger$ iteratively, and compute $\langle\phi_n|\frac{\partial U_n}{\partial a_n}|\psi_{n-1}\rangle$ at each step.

CHAPTER 3
SCALING ANALYSIS AND BENCHMARKING

Having established the analytical gradient expressions and their numerical evaluation in Chapter 2, this chapter presents a systematic comparison of three methods for computing gradients within GRAPE: evaluation of hard-coded gradients (HG), full automatic differentiation (full-AD) [98], and semi-automatic differentiation (semi-AD) [102]. To determine the method most appropriate for a given problem, I inspect the memory usage and runtime scaling of these three approaches. Following this analysis, I present benchmark studies of specific state-transfer and gate-operation problems to illustrate the scaling behavior.

3.1 Scaling analysis of runtime and memory usage

Runtime and memory usage can pose significant challenges when performing optimization tasks for quantum systems with large Hilbert space dimension (d) or for time evolution requiring a significant number of time steps (N). In this section, I analyze the scaling of memory usage and runtime with respect to both N and d, focusing on the cost contribution C_1 for state-transfer and unitary-gate optimization. The findings can be generalized to other cost contributions beyond C_1.

3.1.1 Hard-coded gradients via forward-backward propagation

I start by considering state-transfer problems. The *memory usage* for evaluating ∇C_1^{st} is mainly determined by the required storage of quantum states and the Hamiltonian. In forward-backward propagation, the number of states and instances of Hamiltonian matrices stored at any given time is of the order of unity. Each individual state vector of dimension d consumes memory $\sim \Theta(d)$.

Memory usage for the Hamiltonian depends on the employed storage scheme. In the "worst" case of dense matrix storage, the memory required for the Hamiltonian is $\Theta(d^2)$. For sparse matrix storage, memory usage generally depends on sparsity and is determined by the number of stored Hamiltonian matrix elements (here denoted as κ). For example, for a diagonal or tridiagonal matrix, memory usage is $\kappa \sim \Theta(d)$; for a linear chain of qubits with nearest-neighbor σ_z coupling, it is $\Theta(d \log_2 d)$. Combining the scaling relations above gives an overall asymptotic behavior of memory usage $\Theta(d + \kappa)$.

The *runtime* of the scheme is dominated by the required $\Theta(N)$ evaluations of propagator-state products and their derivative counterparts. Numerically approximating a single propagator-state product requires μ matrix-vector multiplications [see Eq. (2.52)], where the runtime of one instance of matrix-vector multiplication scales as $\Theta(\kappa)$. The implicit dependence of $\mu = \mu(d)$ on d is specific to each particular system and can be difficult to obtain analytically. Numerically, I find for a driven transmon coupled to a cavity that $\mu \sim \Theta(\sqrt{d})$; for a linear chain of coupled qubits, the scaling is $\Theta(\log_2 d)$; see Appendix A for more details. Once combined, the above individual scaling relations lead to an overall runtime asymptotic behavior $\Theta(N\mu\,\kappa)$. For example, in the case of the linear chain of coupled qubits, runtime scales as $\Theta(Nd\,(\log_2 d)^2)$.

For gate-optimization problems, the scaling behavior of memory usage and runtime is as follows. The gradient ∇C_1^g is calculated by sequentially applying forward-backward propagation to each of the d basis states. The memory usage is $\Theta(d + \kappa)$, the same as for ∇C_1^{st}. The runtime scaling is $\Theta(N\mu\kappa d)$, which is d times larger than the runtime scaling for ∇C_1^{st}. (Another way of calculating ∇C_1^g consists of parallelized forward-backward propagation of all basis states. The concrete scaling behavior then depends on the parallelization scheme; that discussion is beyond the scope of this thesis.)

Thanks to the similarities in the expressions and the existence of recursion relations, forward-

backward schemes akin to Fig. 2.2 can also be applied to ∇C_2^{st}, ∇C_3^{st}, and ∇C_3^{g}. As a result, the scaling analysis leads to the same memory usage and runtime requirements as ∇C_1^{st} and ∇C_1^{g}.

3.1.2 Full automatic differentiation

Reverse-mode automatic differentiation extracts gradients by a forward pass and a reverse pass through the computational tree. With the forward pass the cost function C is computed, and the backward pass accumulates the gradient via the chain rule.

The *memory usage* for evaluating ∇C_1^{st} is determined by the requirement that all intermediate numerical results must be stored as part of the forward pass and act as input to the reverse pass. Specifically, as shown in Fig. 2.2, the forward pass evolves the initial state vector to the final state vector in N time steps. The total number of vectors that have to be stored along the way is $\Theta(N\mu)$, since computation of a single short-time propagator-state product via scaling and squaring necessitates μ iterations. In addition, considering the memory usage for the Hamiltonian, the full scaling is $\Theta(N\mu d + \kappa)$.

The *runtime* for evaluating ∇C_1^{st} combines forward and reverse passes. It is dominated by the evaluation of $\Theta(N)$ propagator-state products, each of which has the runtime scaling $\Theta(\mu\kappa)$, leading to an overall runtime scaling of $\Theta(N\mu\kappa)$.

For gate-optimization problems, C_1^{g} is evaluated by evolving not only a single initial state but a whole set of d basis vectors. Evaluating Nd propagator-state products requires $\Theta(N\mu d^2 + \kappa)$ memory, where the number of stored Hamiltonian matrix elements κ takes into account storage of the Hamiltonian. Since κ has the worst-case scaling $\Theta(d^2)$ in the case of a dense Hamiltonian matrix, the overall scaling is $\Theta(N\mu d^2)$. Assuming that the evolution of individual basis vectors is not parallelized, runtime also grows by a factor of d, so that runtime scaling for ∇C_1^{g} is $\Theta(N\mu\kappa d)$.

Despite the specific differences among the expressions of cost contributions C_2^{st}, C_3^{st}, and C_3^{g},

Table 3.1: Memory usage and runtime scaling with respect to Hilbert space dimension d, number of time steps N, the number of stored Hamiltonian matrix elements $\kappa = \kappa(d)$, and the number of matrix-vector multiplications required for evaluating propagator-state products $\mu = \mu(d)$. (The scaling of μ and κ depends on the specific optimization task.) The table contrasts scaling for state transfer and gate operation for hard-coded gradients (HG), full automatic differentiation (full-AD), and semi-automatic differentiation (semi-AD).

	STATE TRANSFER		**GATE OPERATION**	
	Runtime	**Memory usage**	**Runtime**	**Memory usage**
HG	$\Theta(N\mu\kappa)$	$\Theta(d+\kappa)$	$\Theta(N\mu\kappa d)$	$\Theta(d+\kappa)$
Full-AD	$\Theta(N\mu\kappa)$	$\Theta(N\mu d+\kappa)$	$\Theta(N\mu\kappa d)$	$\Theta(N\mu d^2)$
Semi-AD	$\Theta(N\mu\kappa)$	$\Theta(Nd+\kappa)$	$\Theta(N\mu\kappa d)$	$\Theta(Nd^2)$

the scaling of runtime and memory usage is again set by the need to store intermediate vectors and repeatedly perform matrix-vector multiplications. Consequently, the scaling remains the same as for the cost contributions discussed above.

3.1.3 Semi-automatic differentiation

For gradients of cost contributions that are tedious to derive analytically, semi-automatic differentiation [102] offers an alternative approach in which some derivatives are hard-coded and others treated by automatic differentiation. Compared to full-AD, semi-AD has the benefit that the memory usage scaling is independent of μ while maintaining the same runtime scaling.

3.1.4 Comparison

The results of this scaling analysis are summarized in Table 3.1. For full-AD, memory usage grows linearly with respect to μ and number of time steps N. By contrast, memory usage for HG does not increase with μ and N, but remains constant. This advantage is not bought at the cost of worse runtime scaling. Therefore, HG is preferable over full-AD in the optimal control of large

quantum systems where memory usage would otherwise be prohibitively large. Semi-AD presents an interesting compromise viable whenever the memory bottleneck is not too severe. Note that the runtime scaling $\Theta(N\mu\kappa d)$ of HG via scaling and squaring can in principle exceed the scaling $O(Nd^3)$ for matrix exponentiation based on matrix diagonalization (see Appendix B). In that case, the latter is the better option.

I illustrate the discussed scaling by benchmark studies for concrete optimization problems in the following section.

3.2 Benchmark studies

3.2.1 State-transfer benchmark

The first benchmark example studies the following state-transfer problem. Consider a setup in which a driven flux-tunable transmon is capacitively coupled to a superconducting cavity, and perform a state transfer from the cavity vacuum state to the 20-photon Fock state. The Hamiltonian in the frame co-rotating with the transmon drive is

$$H_1(t) = \Delta b^\dagger b + \frac{1}{2}\alpha b^\dagger b(b^\dagger b - 1) + g(cb^\dagger + c^\dagger b) + a_z(t)b^\dagger b + a_x(t)(b + b^\dagger). \tag{3.1}$$

Here, the drive is both longitudinal and transverse; c and b are the annihilation operators for cavity photons and transmon excitations, respectively. α, g, and Δ denote anharmonicity, interaction strength, and the difference between the transmon and cavity frequencies.

Optimization proceeds by minimizing the infidelity C_1^{st} and limiting the occupation of higher transmon levels by including the cost contribution C_2^{st} [Eq. (2.11)]. The latter has the additional benefit of enabling the truncation to only a few transmon levels. I measure runtime and

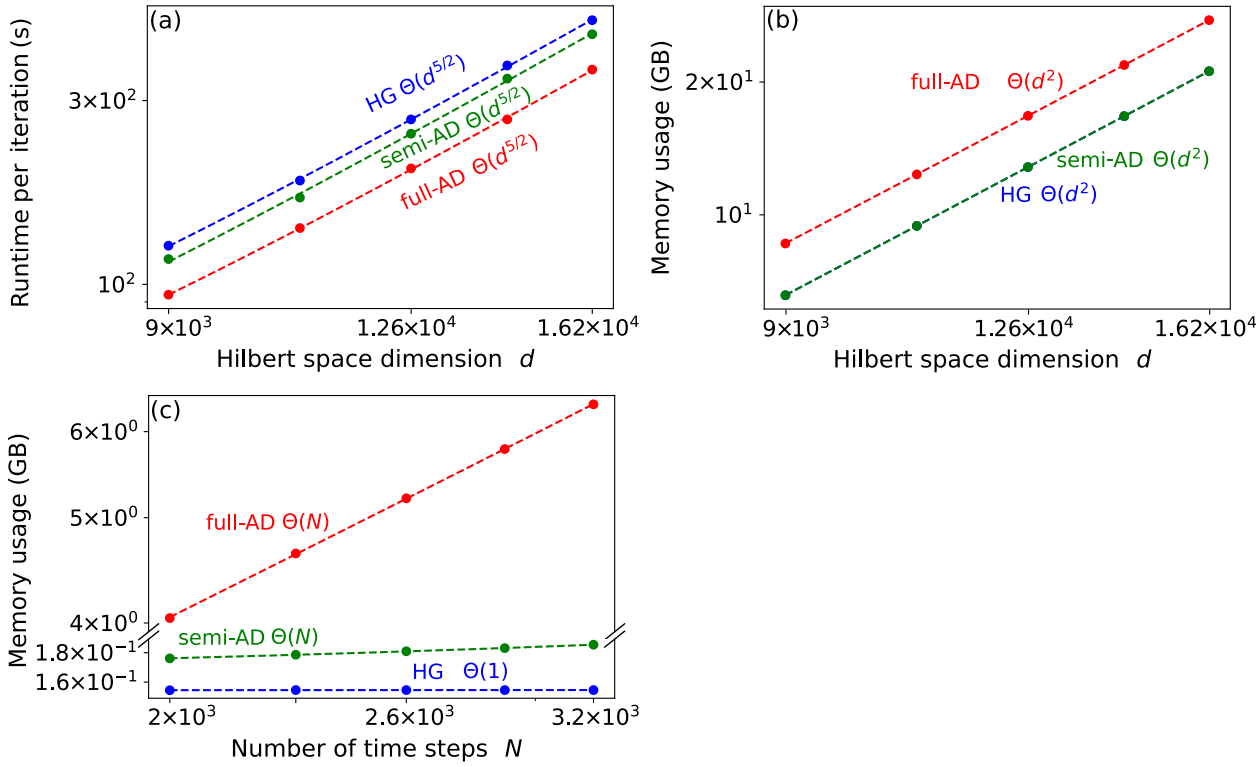

Figure 3.1: Benchmark results for Fock state generation in a system consisting of a cavity coupled to a transmon. (a) Runtime per iteration and (b) memory usage versus Hilbert space dimension d. The dimension is varied by increasing the cavity dimension while fixing the transmon dimension to 6. (c) Memory usage as a function of the number of time steps for $d = 240$. All plots are in log-log scale. Dashed lines are power-law fits. (Parameters: $\Delta/2\pi = 3\,\text{GHz}$, $g/2\pi = 0.1\,\text{GHz}$, $\alpha/2\pi = -0.225\,\text{GHz}$, and constant controls $a_z/2\pi = a_x/2\pi = 0.1\,\text{GHz}$.)

peak memory usage of full-AD,[1] semi-AD, and HG for a single optimization iteration. The runtime is measured by the package `Temci` [113] and the peak memory usage is monitored by the `memory_profiler` package [116]. The optimization is performed on an AMD Ryzen Threadripper 3970X 32-Core CPU with 3.7 GHz frequency.

Benchmark results for the state-transfer optimization are shown in Fig. 3.1. I consider a single

[1] AD is implemented by the package `Autograd` [107] in this benchmark. This package does not support sparse linear algebra, so I implement dense matrix storage for the Hamiltonian in both full-AD and HG for fair comparison.

time step ($N = 1$) and measure the runtime versus d. The results in Fig. 3.1(a) confirm that full-AD, semi-AD, and HG have the same scaling of runtime per time step with respect to d (Table 3.1). The observed runtime scaling $\Theta(d^{5/2})$ is consistent with $\kappa \sim \Theta(d^2)$ due to dense matrix storage for H_1, and $\mu \sim \Theta(d^{1/2})$ (see Appendix A). The scaling of the required memory resources is illustrated in Fig. 3.1(b): memory usage scales with dimension d as $\Theta(d^2)$ for full-AD, semi-AD, and HG, consistent with $\kappa \sim \Theta(d^2)$ being the dominant contribution. As anticipated, the scaling of memory with N differs between full-AD, semi-AD, and HG [Fig. 3.1(c)]. For full-AD and semi-AD, the scaling is linear with N, while it is independent of N for HG. The slope of full-AD is much larger than the one of semi-AD, since the former has extra dependence on μ (Table 3.1), which is a constant ≈ 100 in this benchmark. The results confirm the favorable memory efficiency of HG and semi-AD compared to full-AD in state-transfer problems with large number of time steps.

3.2.2 Gate-operation benchmark

I next turn to an example for optimizing a unitary gate. This benchmark is performed for three driven transmons with nearest-neighbor coupling. The target gate consists of simultaneous Hadamard operations on each of the three transmons. For concreteness, I consider the case of transmons with the same frequency, driven resonantly. The full Hamiltonian in the rotating frame of the drive is

$$H_2(t) = \sum_{\nu=1}^{3} \left[\frac{1}{2}\alpha b_\nu^\dagger b_\nu (b_\nu^\dagger b_\nu - 1) + a^{(\nu)}(t)(b_\nu + b_\nu^\dagger) \right] + g(b_1 b_2^\dagger + b_2 b_3^\dagger + \text{h.c.}). \tag{3.2}$$

The goal of the optimization is to minimize the infidelity C_2^g. The setup for this benchmark is the same as in the previous example.

Fig. 3.2 records the benchmarks based on a single time step ($N = 1$). The observed runtime

scaling with dimension d is $\Theta(d^{11/3})$ for full-AD, semi-AD, and HG, see Fig. 3.2(a). This power law arises from $\Theta(\mu\kappa d)$ for $\kappa \sim \Theta(d^2)$ and $\mu \sim \Theta(d^{2/3})$ [see Appendix A]. The memory usage scaling with d is illustrated in Fig. 3.2(b), confirming that full-AD has an unfavorable extra $\mu \sim \Theta(d^{2/3})$ dependence compared to HG and semi-AD (Table 3.1). The memory usage scaling with N is as expected [Fig. 3.2(c)]: for semi-AD, the scaling is linear with N; for HG, it is independent of N. (The memory usage of full-AD in this case exceeds available resources, and thus is not shown.) Throughout this benchmark, this linear dependence with N causes semi-AD to consume about 10 times more memory than HG. This underlines that HG is preferable whenever memory constraints become a limitation in optimization problems with large Hilbert space dimension and large number of time steps. This improvement is achieved without worsening the runtime scaling.

3.3 Choosing among different optimization strategies

There are several considerations which determine whether full-AD, HG, or semi-AD is the optimal strategy for a given optimization problem, see the decision tree in Fig. 3.3. Whenever the Hilbert space dimension d and the number of time steps N are sufficiently small, memory usage may not be a bottleneck. In this case, full-AD may be preferable, since it eliminates the need for hard-coded gradients. If the memory cost of full-AD is not affordable but analytical gradients are tedious to compute, semi-AD may be a viable compromise, though still less memory efficient than HG. In all other cases, HG is the method of choice, since it can handle optimization with larger N and d compared to full-AD and semi-AD.

The choice of implementing HG via scaling and squaring or matrix diagonalization should be informed by whether the Hamiltonian is sparse and κ scales better than $\Theta(d^2)$. In the latter case, scaling and squaring is preferable and leads to an overall memory usage $\sim \Theta(d+\kappa)$, which is better than the memory scaling $O(d^2)$ of matrix diagonalization. By contrast, for Hamiltonians with

$\kappa \sim \Theta(d^2)$, these two methods are equivalent in memory usage scaling, and the one with better runtime for a specific optimization task should be selected. While the runtime for optimization using matrix diagonalization scales as $O(d^3)$ [Appendix B], the runtime for optimization based on scaling and squaring differs between state-transfer and gate optimizations: (1) for state transfer, the runtime scales as $\Theta(\mu d^2)$; hence, whenever μ scales better than $\Theta(d)$, scaling and squaring wins over matrix diagonalization; (2) for gate optimization, the runtime scales as $\Theta(\mu d^3)$; hence matrix diagonalization is preferable.

The scaling analysis and benchmarks presented in this chapter confirm that hard-coded gradients provide the most memory-efficient approach for GRAPE-based optimization of large quantum systems. In the next chapter, I apply these insights to a concrete problem: designing detuning-robust control pulses for transmon gates.

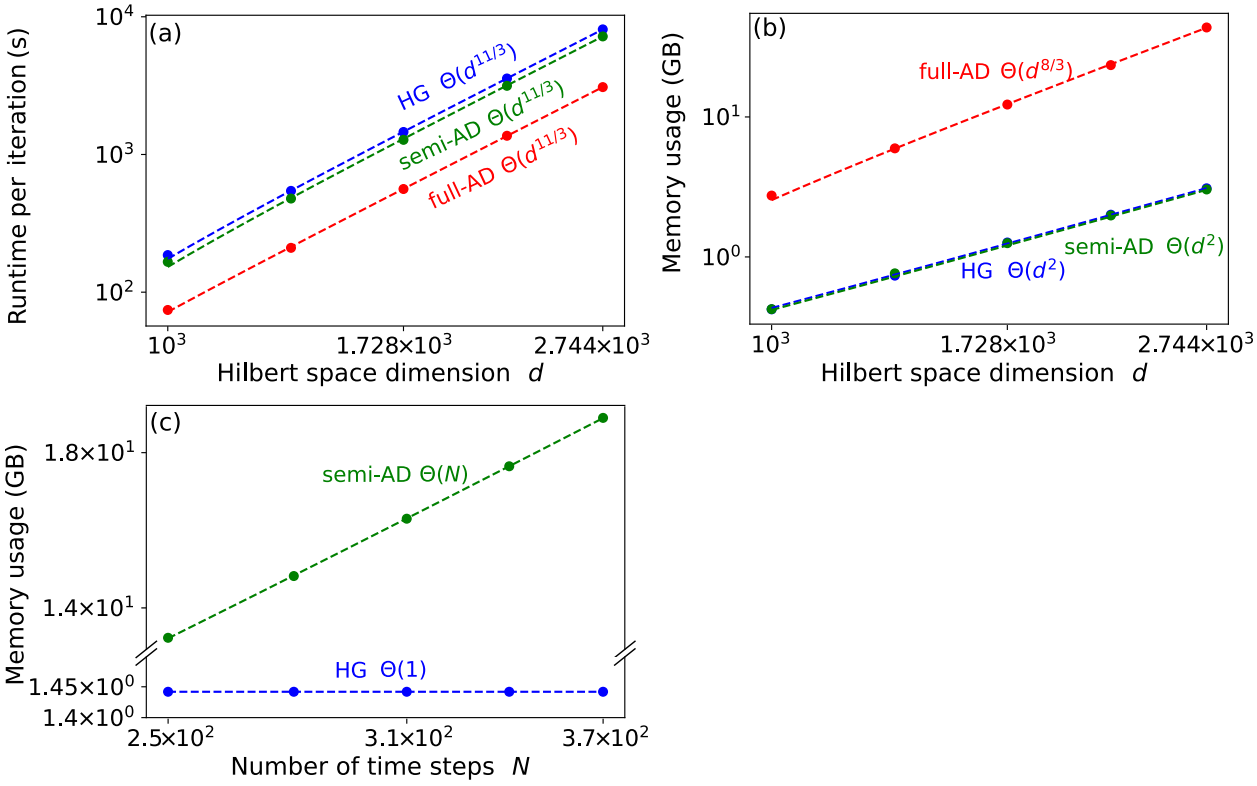

Figure 3.2: Benchmark results for unitary gate optimization in a system consisting of three coupled transmons. (a) Runtime per iteration and (b) memory usage versus Hilbert space dimension d. The total Hilbert space dimension d is increased by enlarging the cutoff of each transmon from 10 to 14 simultaneously. (c) Memory usage as a function of the number of time steps for $d = 12^3 = 1728$. Full-AD (not shown) would require more memory than available resources (estimated: 1 TB). All plots are in log-log scale. Dashed lines are power-law fits. (Parameters: $g/2\pi = 0.1$ GHz, $\alpha/2\pi = -0.225$ GHz, and $a^{(\nu)}/2\pi = 0.1$ GHz.)

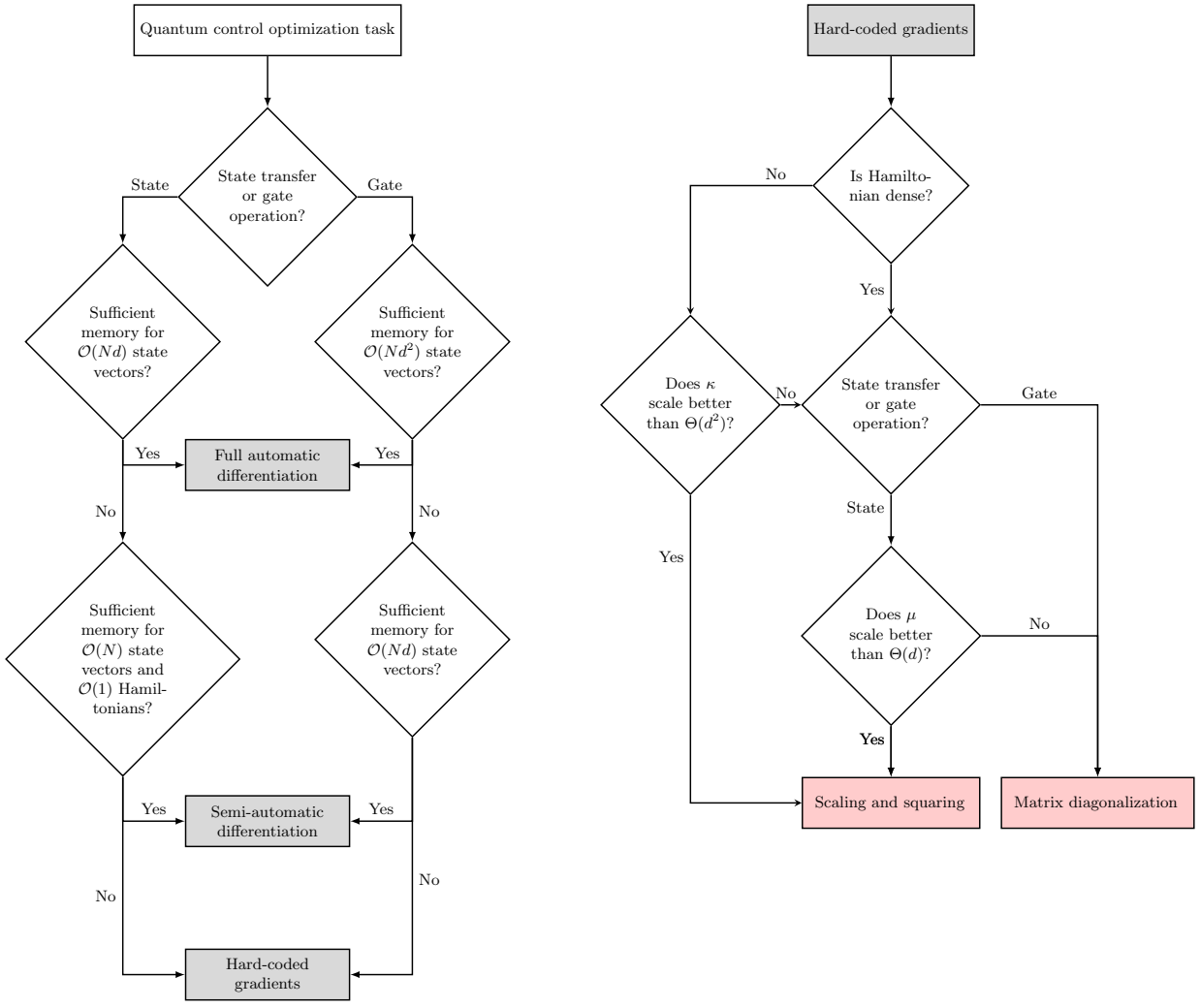

Figure 3.3: Decision trees for selecting the optimal numerical strategy. (a) Choosing among three numerical methods for evaluating gradients: full automatic differentiation, semi-automatic differentiation, hard-coded analytical gradients. (Here, $\mu \sim \Theta(d)$ is assumed; for different scaling see Table 3.1.) (b) Determining the appropriate strategy for evaluating propagator-state products.

CHAPTER 4
APPLICATION OF GRAPE: ROBUST CONTROL FOR TRANSMON

This chapter turns to a concrete application of GRAPE: the design of detuning-robust control pulses for transmon gates. The transmon has been widely used as either a qubit or an ancilla for bosonic qubits. In both cases, the $\hat{X}_\theta$ gate in the first two-level subspace of the transmon is crucial for realizing universal gates in the composite system. One important preliminary to obtaining high-fidelity gates in the composite system is to acquire a robust $\hat{X}_\theta$ in the single-transmon system.

Detuning-robust control for two-level systems has been extensively investigated using various approaches, including reverse engineering [117, 118], space curve quantum control [119], pulse engineering [119–127], and the collocation method [128, 129]. However, protocols developed for two-level systems cannot be directly applied to weakly anharmonic transmons due to leakage out of the qubit subspace. One promising solution is to apply the DRAG scheme to robust single-quadrature pulses developed for two-level systems [130]. Additionally, Ref. [131] directly optimizes two-quadrature pulses for transmons. In the following, I introduce an alternative optimization-based procedure to design detuning-robust control for transmon $\hat{X}_\theta$ gates and benchmark its performance.

4.1 Metrics of robust control

During the gate operation, the Hamiltonian of interest may depend on an unknown quantity δ. This uncertainty can cause the realized gate to deviate from the target gate, thus generally lowering gate fidelity $\mathcal{I}(\delta)$. To quantify the robustness of the infidelity against δ, I consider both the mean

infidelity and its standard deviation as key metrics,

$$\mathcal{I}_\mu = \int_{\mathbb{D}} \mathrm{d}\delta\, p(\delta)\mathcal{I}(\delta), \quad \mathcal{I}_\sigma = \sqrt{\int_{\mathbb{D}} \mathrm{d}\delta\, p(\delta)\big[\mathcal{I}(\delta) - I_\mu\big]^2}. \tag{4.1}$$

Here, the probability distribution $p(\delta)$ satisfies $\int_{\mathbb{D}} \mathrm{d}\delta\, p(\delta) = 1$ for a specified range $\mathbb{D}$.

In the case of interest, the Hamiltonian governing the transmon qubit in the co-rotating frame of the drive frequency is

$$\hat{H}_{\text{trans}}(t) = K_{\text{q}}\hat{q}^{\dagger 2}\hat{q}^2/2 + \varepsilon_I(t)(\hat{q}^\dagger + \hat{q}) + i\varepsilon_Q(t)(\hat{q}^\dagger - \hat{q}) + \delta\hat{q}^\dagger\hat{q}. \tag{4.2}$$

Here, $\varepsilon_I(t)$ and $\varepsilon_Q(t)$ describe control envelopes in the I and Q quadratures, and δ denotes the difference between the drive frequency and the unknown shifted qubit frequency.The closed-system gate infidelity follows

$$\mathcal{I}_{\text{c}}(\delta) = 1 - \frac{1}{4}\left|\mathrm{Tr}\left[\hat{P}_{0,1}\hat{U}^\dagger_{\text{target}}\hat{U}_\delta(t_{\text{g}})\right]\right|^2, \tag{4.3}$$

where $\hat{P}_{0,1}$ is a projector onto the computational subspace formed by the lowest two eigenstates. Here, $\hat{U}_{\text{target}}$ is the target unitary on the transmon, and $\hat{U}_\delta(t_{\text{g}}) = \mathcal{T}\exp\Big[-i\int_0^{t_{\text{g}}}\mathrm{d}\tau\,\hat{H}_{\text{trans}}(\tau)\Big]$ is the actual unitary achieved over a gate duration t_{g}, where $\mathcal{T}$ is the time-ordering operator.

In the presence of decoherence, the open-system infidelity follows [132]

$$\mathcal{I}_{\text{o}}(\delta) = 1 - \frac{1}{4}\mathrm{Tr}\Big[\mathcal{P}_{0,1}\mathcal{L}^\dagger_{\text{target}}\mathcal{L}_\delta(t_{\text{g}})\Big]. \tag{4.4}$$

Here, the density matrix is vectorized by stacking the elements row-by-row. $\mathcal{P}_{0,1} = \hat{P}_{0,1}\otimes\hat{P}_{0,1}$ is the superprojector onto the computational subspace, $\mathcal{L}_{\text{target}} = \hat{U}_{\text{target}}\otimes\hat{U}^*_{\text{target}}$ is the target superoperator, and $\mathcal{L}_\delta(t_{\text{g}})$ is the superoperator realized by the Lindblad master equation. Note that in the absence of decoherence, $\mathcal{L}_\delta(t_{\text{g}}) = \hat{U}_\delta(t_{\text{g}})\otimes\hat{U}^*_\delta(t_{\text{g}})$.

4.2 Optimization procedure

Quantum optimal control has been extensively used to develop robust control schemes by minimizing cost functions that encode robustness metrics. One approach is to penalize the derivative of the closed- or open-system fidelity with respect to detuning δ [130, 131, 133]. However, a more intuitive and pragmatic method is to penalize the average infidelity [134–138]

$$C_{\mathrm{o,c}} = \frac{1}{N}\sum_{i=1}^{N}\mathcal{I}_{\mathrm{o,c}}(\delta_i). \tag{4.5}$$

Here, δ_i denotes sample values of detunings within the specified range of $\mathbb{D}$. Minimization of the above cost function can suppress both mean infidelity and its standard deviation, as evidenced by the subsequent numerical results.

To minimize the cost function, I optimize the pulses, including both the envelopes $\varepsilon_{I,Q}(t)$ and the pulse duration t_{g}. An optimal duration is determined by balancing two competing factors: a shorter duration leads to unwanted leakage, whereas a longer duration results in decoherence errors. A direct approach to optimize pulses involves using an open-system optimizer for various durations and selecting the one that minimizes the cost function C_{o}. However, such optimization requires time-intensive open-system simulations. Instead, I propose a method that, while not guaranteed to be equivalent, yields promising results. The method consists of two steps: (1) employing closed-system optimization to minimize C_{c} for each potential duration, and (2) with given decoherence rates, evaluating C_{o} for the optimized pulses from the first step, then selecting the pulse with the lowest C_{o}. The approach is summarized in Fig. 4.1.

For the closed-system optimization, I use Gradient Ascent Pulse Engineering (GRAPE), a method widely employed for optimal control in various contexts [137, 139, 140]. The gradients required in GRAPE are calculated by using automatic differentiation, thereby eliminating the need

for manually coding the analytical gradients of the cost function [141–143]. The initial pulse ansatz is

$$\begin{aligned}\varepsilon_I^{\text{initial}}(t) = \frac{1}{t_{\text{g}}}\big[&(a-\theta/2)\cos(2\pi t/t_{\text{g}}) + (b-a)\cos(4\pi t/t_{\text{g}}) \\ &+ (c-b)\cos(6\pi t/t_{\text{g}}) - c\cos(8\pi t/t_{\text{g}}) + \theta/2\big], \\ \varepsilon_Q^{\text{initial}}(t) = &-\dot{\varepsilon}_I^{\text{initial}}(t)/K_{\text{q}},\end{aligned} \tag{4.6}$$

where θ is the rotation angle for the target $\hat{X}_\theta$ gate ($\theta = \pi, \pi/2$ in this study). This ansatz for the I-quadrature control has been used to obtain robust control for two-level systems [127]. The initial guess for the Q-quadrature is inspired by the DRAG pulse scheme [144] to suppress leakage. The pulse is then optimized starting from various initial guesses for parameters $a, b, c \in \{-10, -8, -6, \cdots, 10\}$, and the pulse with the lowest cost value C_{c} is selected. This process is accelerated by parallelizing with different initial values.

Throughout the closed-system optimization process, I ensure pulse smoothness by filtering out frequency components that exceed maximum allowed bandwidths of $5/t_{\text{g}}$ and $10/t_{\text{g}}$ for $\varepsilon_I(t)$ and $\varepsilon_Q(t)$, respectively. These empirical values effectively balance pulse smoothness and the level of robustness required.

4.3 Benchmarking and comparison

I benchmark the optimization procedure for a transmon with typical parameters: $K_{\text{q}}/2\pi = -200\,\text{MHz}$, $T_1 = 50\,\mu\text{s}$, and $T_\phi = 50\,\mu\text{s}$. The optimization results for the $\hat{X}_\pi$ gate are shown in Fig. 4.2. Figure 4.2(a) shows the average infidelities obtained for closed- and open-system simulations with varying pulse durations. Without decoherence, extending the pulse duration generally leads to a reduction in the cost value. In contrast, in the presence of decoherence, utilizing shorter pulses is

more effective in reducing the infidelity. Given the particular transmon parameters under consideration, a pulse duration of 20 ns achieves the lowest cost. The temporal profiles and frequency components of the optimized 20-ns pulse are shown in Fig. 4.2(b) and (c), respectively. In Fig. 4.2(d), the infidelity as a function of detuning is studied, comparing the performance between the optimized and the DRAG pulses. In both closed- and open-system simulations, the optimized pulse consistently demonstrates substantially reduced infidelities across almost all detunings. Consequently, the optimized pulse exhibits enhanced robustness against frequency detuning in the transmon ancilla. The open-system infidelities obtained from these two pulses are limited by distinct factors: the infidelity derived from the QOC pulse is dominated by decoherence errors, while the one generated from the DRAG pulse is limited by coherent error.

I then compare the robustness of the pulses derived from this optimization procedure with selected recent literature. First, following the methodology outlined in Ref. [131], I obtain robust pulses using the `Q-CTRL` package [125, 145]. To distinguish between these results and those generated by the optimization procedure presented here, I use the label "Q-CTRL" for the former and "QOC" for the latter. The averaged infidelities [Eq. (4.5)] of the closed system as a function of pulse duration are shown in Fig. 4.3(a). While the average infidelities for Q-CTRL (red) and QOC (blue) are similar for shorter durations, QOC demonstrates orders of magnitude lower infidelity for longer durations. To illustrate the robustness details of the Q-CTRL results, I examine the 20-ns case, which exhibits the lowest infidelity. The I and Q quadratures of the pulse envelopes and their corresponding Fourier components are shown in Fig. 4.3(b) and (c). The infidelity as a function of detuning is presented in Fig. 4.3(d). For a closed system, the Q-CTRL results (red dashed line) show lower infidelity near resonance but exhibit higher infidelity off-resonance ($\delta/2\pi > 5\,\text{MHz}$) compared to the QOC results (blue dashed line). This difference likely arises from the different cost functions used in the two optimizations: Q-CTRL penalizes the derivative of the infidelity at zero

detuning, whereas QOC minimizes the average infidelity over a sampling of detunings ($\delta_i/2\pi \in \{\pm 1, \pm 3, \pm 4, \pm 7, \pm 10\}$ MHz in this example). Notably, in the presence of ancilla decoherence ($T_1 = 50\,\mu$s, $T_\phi = 50\,\mu$s), the infidelity at small detuning is dominated by incoherent error. This makes QOC the overall better choice within the considered range of detuning for robust pulses.

The results of this chapter demonstrate that the GRAPE-based optimization procedure developed here generates detuning-robust transmon pulses that outperform both conventional DRAG pulses and Q-CTRL pulses across the relevant detuning range.

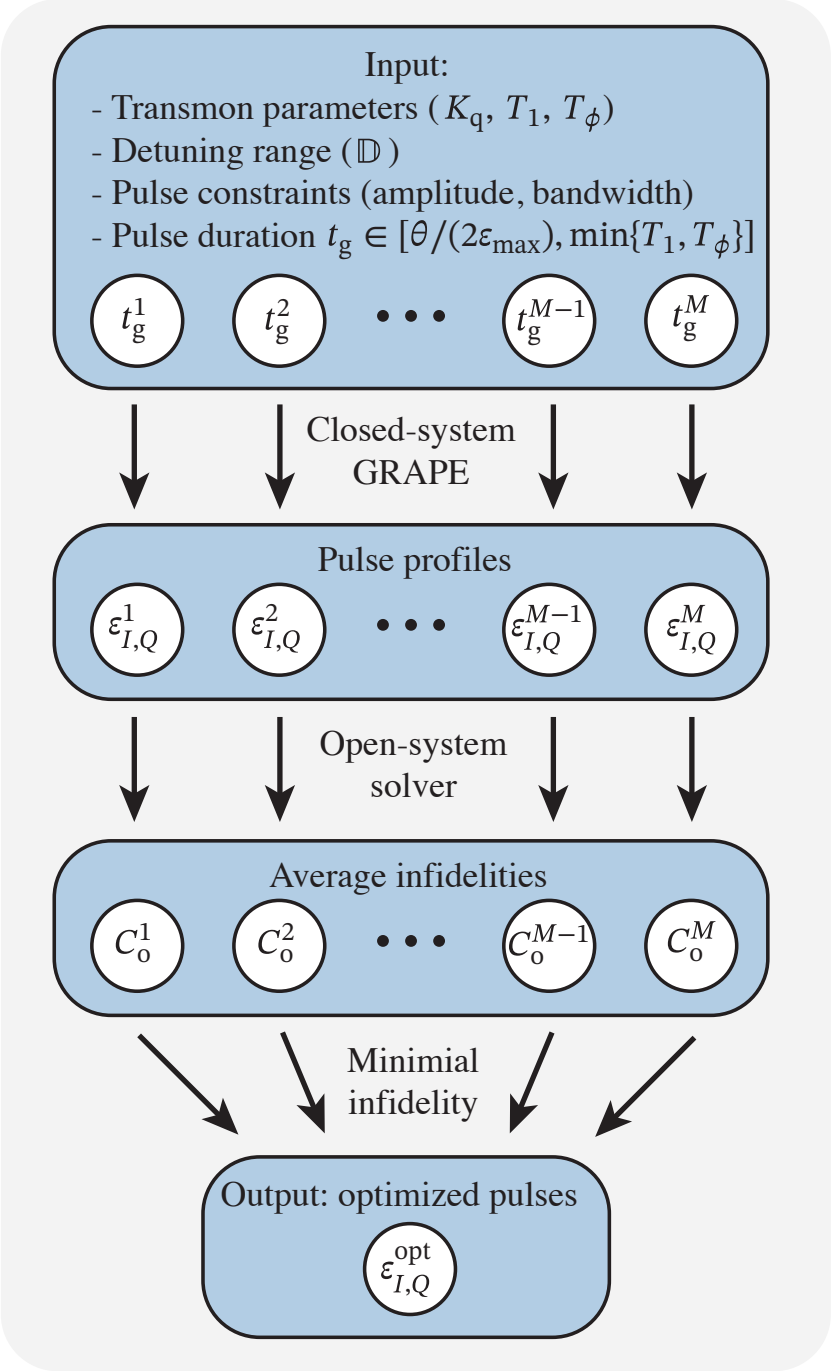

Figure 4.1: Optimization procedure to obtain detuning-robust $\hat{X}_\theta$ gates in transmons. Input includes pulse constraints, transmon parameters, and a set of M pulse durations. The choice of duration should be shorter than the decay time T_1 and longer than $1/\varepsilon_{\max}$, where $\varepsilon_{\max}$ is the maximum pulse amplitude. Closed-system GRAPE is performed for each duration to obtain the corresponding pulse profiles. Subsequently, an open-system solver evaluates the cost function C_o for pulses with varying durations, selecting the pulse that minimizes C_o.

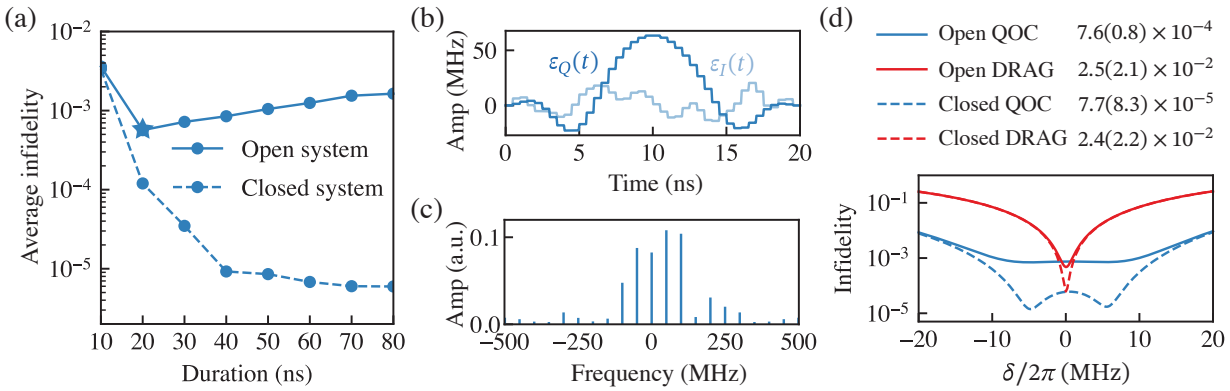

Figure 4.2: Optimization results for the $\hat{X}_\pi$ gate. (a) Average infidelities in Eq. (4.5) versus pulse durations for both closed (dashed line) and open (solid line) systems. The optimal duration selected from the range considered for this optimization is 20 ns (indicated by a star). (b) I and Q quadratures of the pulse envelopes for the selected 20-ns pulse from (a). (c) The frequency components of the pulse envelopes, corresponding to the 20-ns pulse. (d) Comparative analysis of the 20-ns pulses, from both QOC (blue) and DRAG (red) pulses, in terms of their infidelities as a function of detuning. The average and standard deviation of the infidelities are tabulated correspondingly by considering a uniform distribution of $\rho(\delta)$ for simplicity. The result shows that the optimized pulse is more robust than the commonly used DRAG pulse. [Parameters: $\delta_i/2\pi \in \{\pm1, \pm3, \pm4, \pm7, \pm10\}$ MHz, $K_\mathrm{q}/2\pi = -200$ MHz, $T_1 = 50\,\mu$s, $T_\phi = 50\,\mu$s.]

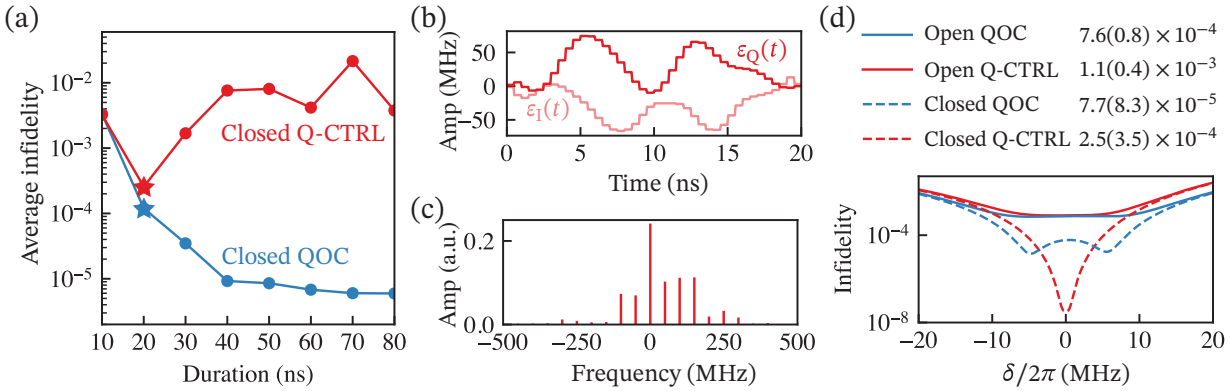

Figure 4.3: Comparison of optimization results between Q-CTRL and QOC for the $\hat{X}_\pi$ gate. (a) Average infidelities of a closed system [Eq. (4.5)] versus pulse durations derived from Q-CTRL (red) and QOC (blue). The optimal duration selected for Q-CTRL from the range considered is 20 ns (indicated by a red star). (b) I and Q quadratures of the Q-CTRL pulse envelopes for the selected 20-ns pulse from (a). (c) Frequency components of the Q-CTRL pulse envelopes corresponding to the 20-ns pulse. (d) Comparative analysis of the 20-ns pulses from both QOC (blue) and Q-CTRL (red) in terms of their infidelities as a function of detuning. The average and standard deviation of the infidelities are tabulated considering a uniform distribution of $\rho(\delta)$ for simplicity. [Parameters: $\delta_i/2\pi \in \{\pm1, \pm3, \pm4, \pm7, \pm10\}$ MHz, $K_q/2\pi = -200$ MHz, $T_1 = 50\,\mu$s, $T_\phi = 50\,\mu$s.]

CHAPTER 5
CONCLUSION AND FUTURE WORK

5.1 Summary of key findings and significance

This thesis has addressed both the numerical implementation and application of Gradient Ascent Pulse Engineering (GRAPE) for quantum optimal control. The key findings are as follows.

In Chapter 2, I revisited the hard-coded gradient (HG) scheme for GRAPE and presented a complete analytical and numerical framework for evaluating cost function gradients. The scaling-and-squaring method for state propagation was developed with rigorous error bounds that account for both truncation and rounding errors, yielding superior numerical stability compared to existing implementations such as `Scipy`'s `expm_multiply`. The auxiliary matrix method was employed to efficiently compute propagator derivatives, and the forward-backward propagation scheme was shown to achieve memory cost independent of the number of time steps.

In Chapter 3, I performed a systematic scaling analysis comparing HG, full-AD, and semi-AD in terms of runtime and memory usage. The analysis demonstrated that HG achieves the same runtime scaling as AD-based methods while requiring significantly less memory—memory usage for HG does not grow with the number of time steps N or the number of matrix-vector multiplications μ. Benchmark studies on state-transfer and gate-optimization problems confirmed these scaling predictions. A decision tree was developed to guide practitioners in choosing the optimal numerical strategy based on problem-specific parameters.

In Chapter 4, I applied GRAPE to design detuning-robust control pulses for transmon $\hat{X}_\theta$ gates. The proposed two-step optimization procedure—closed-system GRAPE followed by open-system

evaluation—was shown to generate pulses that outperform both conventional DRAG pulses and Q-CTRL pulses in terms of robustness against frequency detuning.

5.2 Limitations

The scaling-and-squaring method relies on the choice of truncation order and scaling factor, which are selected heuristically rather than by global optimization of the product ms. While the heuristic performs well in practice, it may not always yield the most efficient computation. The robust pulse optimization procedure assumes a specific detuning distribution and a fixed set of sample detuning values, which may not capture all realistic scenarios.

5.3 Opportunities for future research

Several directions for future work emerge from this thesis. The hard-coded gradient framework could be extended to support more general cost functions and constraints, including those arising in open-system optimization. The decision tree could be refined to incorporate parallelization strategies and GPU acceleration. On the application side, the detuning-robust pulses developed here could be integrated into multi-qubit gate protocols and tested experimentally on superconducting quantum processors.

BIBLIOGRAPHY

[1] R. P. Feynman, International Journal of Theoretical Physics (1982).

[2] S. Lloyd, Science (1996).

[3] Y. Cao *et al.*, Chemical Reviews (2019).

[4] S. McArdle *et al.*, Reviews of Modern Physics (2020).

[5] I. M. Georgescu *et al.*, Reviews of Modern Physics (2014).

[6] U.-J. Wiese, Annalen der Physik (2013).

[7] C. Outeiral *et al.*, WIREs Computational Molecular Science (2021).

[8] R. Santagati *et al.*, Nature Physics (2024).

[9] P. W. Shor, in *Proceedings of the 35th Annual Symposium on Foundations of Computer Science* (1994) pp. 124–134.

[10] A. M. Dalzell, S. McArdle, M. Berta, P. Bienias, C.-F. Chen, A. Gilyén, C. T. Hann, M. J. Kastoryano, E. T. Khabiboulline, A. Kubica, G. Salton, S. Wang, and F. G. S. L. Brandão, *Quantum Algorithms: A Survey of Applications and End-to-end Complexities* (Cambridge University Press, 2025).

[11] National Institute of Standards and Technology, Nist releases first 3 finalized post-quantum encryption standards, `https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards` (2024), accessed 2026-03-15.

[12] L. K. Grover, in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing* (1996) pp. 212–219.

[13] A. Montanaro, npj Quantum Information (2016).

[14] R. Orús *et al.*, Reviews in Physics (2019).

[15] D. Herman *et al.*, Nature Reviews Physics (2023).

[16] J. Biamonte *et al.*, Nature (2017).

[17] K. Blekos *et al.*, Physics Reports (2024).

[18] Nature Physics, Nature Physics (2023).

[19] H. Soller, M. Gschwendtner, S. Shabani, and W. Svejstrup, *The Year of Quantum: From Concept to Reality in 2025*, Tech. Rep. (McKinsey & Company, 2025).

[20] D. P. DiVincenzo, Fortschritte der Physik (2000).

[21] T. D. Ladd *et al.*, Nature (2010).

[22] M. W. Doherty *et al.*, Physics Reports (2013).

[23] S. Pezzagna *et al.*, Applied Physics Reviews (2021).

[24] C. Monroe *et al.*, Science (2013).

[25] C. D. Bruzewicz *et al.*, Applied Physics Reviews (2019).

[26] M. Saffman, Journal of Physics B: Atomic, Molecular and Optical Physics (2016).

[27] D. Bluvstein *et al.*, Nature (2022).

[28] P. Kok *et al.*, Reviews of Modern Physics (2007).

[29] S. Slussarenko *et al.*, Applied Physics Reviews (2019).

[30] P. Krantz *et al.*, Applied Physics Reviews (2019).

[31] M. Kjaergaard *et al.*, Annual Review of Condensed Matter Physics (2020).

[32] IBM, Ibm quantum roadmap (2025), accessed 2026-03-15.

[33] Google Quantum AI, Roadmap (2026), accessed 2026-03-15.

[34] Google Quantum AI and Collaborators, Nature (2025).

[35] Y. Nakamura *et al.*, Nature (1999).

[36] Y. A. Pashkin *et al.*, Quantum Information Processing (2009).

[37] J. E. Mooij *et al.*, Science (1999).

[38] F. Yan *et al.*, Nature Communications (2016).

[39] J. Koch *et al.*, Physical Review A (2007).

[40] A. A. Houck *et al.*, Quantum Information Processing (2009).

[41] V. E. Manucharyan *et al.*, Science (2009).

[42] H. Zhang *et al.*, Physical Review X (2021).

[43] K. Kalashnikov *et al.*, PRX Quantum (2020).

[44] A. Gyenis *et al.*, PRX Quantum (2021).

[45] A. Kitaev, Protected qubit based on a superconducting current mirror (2006).

[46] W. C. Smith *et al.*, npj Quantum Information (2020).

[47] Y.-Y. Jiang *et al.*, National Science Review (2025).

[48] IBM Quantum, Ibm quantum system two: The era of quantum utility is here (2023), introduces the 1,121-qubit Condor processor.

[49] A. Krasnok *et al.*, Applied Physics Reviews (2024).

[50] A. Copetudo *et al.*, Applied Physics Letters (2024).

[51] H. Paik *et al.*, Physical Review Letters (2011).

[52] M. Reagor *et al.*, Applied Physics Letters (2013).

[53] M. Reagor *et al.*, Physical Review B (2016).

[54] O. Milul *et al.*, PRX Quantum (2023).

[55] A. Romanenko *et al.*, Physical Review Applied (2020).

[56] H. Wang *et al.*, Applied Physics Letters (2009).

[57] A. Megrant *et al.*, Applied Physics Letters (2012).

[58] S. Ganjam *et al.*, Nature Communications (2024).

[59] L. Krayzman *et al.*, Applied Physics Letters (2024).

[60] A. Joshi *et al.*, Quantum Science and Technology (2021).

[61] W. Cai *et al.*, Fundamental Research (2021).

[62] A. Blais *et al.*, Reviews of Modern Physics (2021).

[63] W.-L. Ma *et al.*, Science Bulletin (2021).

[64] D. I. Schuster *et al.*, Nature (2007).

[65] R. W. Heeres *et al.*, Physical Review Letters (2015).

[66] S. Krastanov *et al.*, Physical Review A (2015).

[67] X. You *et al.*, Physical Review Applied (2024).

[68] A. Eickbusch *et al.*, Nature Physics (2022).

[69] A. A. Diringer *et al.*, Physical Review X (2024).

[70] M. Kudra *et al.*, PRX Quantum (2022).

[71] J. Landgraf *et al.*, Physical Review Letters (2024).

[72] Y. Y. Gao *et al.*, Physical Review X (2018).

[73] Y. Zhang *et al.*, Physical Review A (2019).

[74] B. J. Chapman *et al.*, PRX Quantum (2023).

[75] Y. Lu *et al.*, Nature Communications (2023).

[76] A. Maiti *et al.*, PRX Quantum (2025).

[77] I. Pietikäinen *et al.*, PRX Quantum (2024).

[78] J. D. Teoh *et al.*, Proceedings of the National Academy of Sciences of the United States of America (2023).

[79] T. Tsunoda *et al.*, PRX Quantum (2023).

[80] Z. Leghtas *et al.*, Physical Review Letters (2013).

[81] M. Mirrahimi *et al.*, New Journal of Physics (2014).

[82] M. H. Michael *et al.*, Physical Review X (2016).

[83] V. V. Albert *et al.*, Physical Review A (2018).

[84] D. Gottesman *et al.*, Physical Review A (2001).

[85] A. J. Brady *et al.*, Progress in Quantum Electronics (2024).

[86] N. Ofek *et al.*, Nature (2016).

[87] P. Campagne-Ibarcq *et al.*, Nature (2020).

[88] V. V. Sivak *et al.*, Nature (2023).

[89] Z. Ni *et al.*, Nature (2023).

[90] J. M. Gertler *et al.*, Nature (2021).

[91] R. Lescanne *et al.*, Nature Physics (2020).

[92] P. Leviant *et al.*, Quantum (2022).

[93] R. Shillito *et al.*, Physical Review Applied (2022).

[94] M. F. Dumas *et al.*, Physical Review X (2024).

[95] C. P. Koch *et al.*, EPJ Quantum Technology (2022).

[96] Y. Lu *et al.*, Journal of Physics Communications (2024).

[97] A. G. Baydin *et al.*, Journal of Machine Learning Research **18**, 1 (2018).

[98] N. Leung *et al.*, Physical Review A **95**, 042318 (2017).

[99] T. Propson, Qoc, `https://github.com/SchusterLab/qoc.git` (2019).

[100] M. Abdelhafez *et al.*, Physical Review A **99**, 052327 (2019).

[101] M. Zhang *et al.*, Phys. Rev. Res. **4**, 043057 (2022).

[102] M. H. Goerz *et al.*, Quantum **6**, 871 (2022).

[103] F. Arute *et al.*, Nature **574**, 505 (2019).

[104] M. Gong *et al.*, Science **372**, 948 (2021).

[105] N. Khaneja *et al.*, Journal of Magnetic Resonance **172**, 296 (2005).

[106] M. Abadi *et al.*, TensorFlow: Large-scale machine learning on heterogeneous systems (2015), software available from tensorflow.org.

[107] D. Maclaurin *et al.*, in *ICML 2015 AutoML workshop*, Vol. 238 (2015).

[108] C. Moler *et al.*, SIAM Review **45**, 3 (2003).

[109] B. Codenotti *et al.*, Calcolo **29**, 1 (1992).

[110] N. J. Higham, *Accuracy and stability of numerical algorithms*, 4th ed. (SIAM, 2002) Chap. 2–3.

[111] J. H. Wilkinson, *Rounding errors in algebraic processes,* (Courier Corporation, 1994) p. 82.

[112] A. H. Al-Mohy *et al.*, SIAM Journal on Scientific Computing **33**, 488 (2011).

[113] J. Bechberger, Temci, `https://github.com/parttimenerd/temci.git` (2015).

[114] D. Goodwin *et al.*, The Journal of Chemical Physics **143**, 084113 (2015).

[115] I. Najfeld *et al.*, Advances in Applied Mathematics **16**, 321 (1995).

[116] F. Pedregosa, Memory profiler, `https://github.com/pythonprofilers/memory_profiler.git` (2012).

[117] D. Daems *et al.*, Phys. Rev. Lett. **111**, 050404 (2013).

[118] G. Dridi *et al.*, Phys. Rev. Lett. **125**, 250403 (2020).

[119] E. Barnes *et al.*, Quantum Sci. Technol. **7**, 023001 (2022).

[120] X. Wang *et al.*, Nat. Commun. **3**, 997 (2012).

[121] B. T. Torosov *et al.*, Phys. Rev. A **83**, 053420 (2011).

[122] P. Owrutsky *et al.*, Phys. Rev. A **86**, 022315 (2012).

[123] R. Tycko *et al.*, Phys. Rev. Lett. **51**, 775 (1983).

[124] H.-N. Wu *et al.*, Phys. Rev. A **107**, 023103 (2023).

[125] H. Ball *et al.*, Quantum Sci. Technol. **6**, 044011 (2021).

[126] S. Wimperis *et al.*, J. Magn. Reson. A **109**, 221 (1994).

[127] S. Pasini *et al.*, Phys. Rev. A **80**, 022328 (2009).

[128] T. Propson *et al.*, Phys. Rev. Appl. **17**, 014036 (2022).

[129] A. Trowbridge *et al.*, arXiv:2305.03261 (2023).

[130] Y.-J. Hai *et al.*, arXiv:2210.14521 (2023).

[131] A. R. R. Carvalho *et al.*, Phys. Rev. Appl. **15**, 064054 (2021).

[132] M. R. Abdelhafez, *Quantum Optimal Control Using Automatic Differentiation*, Ph.D. thesis, The University of Chicago (2019).

[133] S. Watanabe *et al.*, Phys. Rev. A **109**, 012616 (2024).

[134] P. Reinhold, *Controlling error-correctable bosonic qubits*, Ph.D. thesis, Yale Univ. (2019).

[135] J. Allen, *Robust Optimal Control of the Cross-Resonance Gate in Superconducting Qubits*, Ph.D. thesis, Univ. of Surrey (2020).

[136] R. L. Kosut *et al.*, Phys. Rev. A **88**, 052326 (2013).

[137] N. Khaneja *et al.*, J. Magn. Reson. **172**, 296 (2005).

[138] P. Rembold *et al.*, AVS Quantum Sci. **2**, 024701 (2020).

[139] Y. Lu *et al.*, J. Phys. Commun. **8**, 025002 (2024).

[140] C. P. Koch *et al.*, EPJ Quantum Technol. **9**, 19 (2022).

[141] A. G. Baydin *et al.*, arXiv:1404.7456 (2014).

[142] N. Leung *et al.*, Phys. Rev. A **95**, 042318 (2017).

[143] M. Abdelhafez *et al.*, Phys. Rev. A **99**, 052327 (2019).

[144] F. Motzoi *et al.*, Phys. Rev. Lett. **103**, 110501 (2009).

[145] Q-CTRL, Boulder opal (2022).

[146] V. Y. Pan *et al.*, in *Proceedings of the Thirty-First Annual ACM Symposium on Theory of Computing*, STOC '99 (Association for Computing Machinery, 1999) p. 507–516.

OPTIMIZING PULSE FOR GATE AND DECOHERENCE FOR SUPERCONDUCTING QUBITS

Approved by:

Gustave Flaubert
Romanticism
France

Emile Zola
Naturalism
France

Ivan Turgenev
Realism
Russia

Date Approved: August 1, 2021

APPENDIX A

SCALING OF THE NUMBER OF MATRIX-VECTOR MULTIPLICATIONS μ

The asymptotic behavior of runtime, discussed in Chapter 3, is governed by the scaling of μ (2.52) with the dimension d of the Hilbert space. The dependence $\mu(d)$ is specific to the matrix A to be exponentiated (which is proportional to the Hamiltonian). This prevents a general scaling relation from being stated. I thus turn to a discussion of several concrete examples.

I first consider the example of the transmon–cavity system, see Eq. (3.1), where d is increased via the cavity dimension d_c while leaving the transmon cutoff fixed. The corresponding Hamiltonian H_1, written in the bare product basis, then has a constant maximum number of non-zero elements in each row. (I.e., the parameter σ' (2.42) is independent of d.) Numerically, I find in this case that μ scales linearly with $||A||_1$ $\big(\mu \sim \Theta(||A||_1)\big)$, and this linearity holds for a wide range of τ, see Fig. A.1(a). In this example, two ingredients lead to the scaling $||A||_1 \sim \Theta(d^{1/2})$: (1) d is a constant multiple of d_c, and (2) the cavity ladder operators in H_1 which have a one-norm scaling $\Theta(d_c^{1/2})$. Thus, the overall scaling $\mu \sim \Theta(d^{1/2})$ is obtained, see Fig. A.1(e). In the second example, I consider the system of three coupled transmons, see Eq. (3.2). The Hamiltonian H_2 is written in the harmonic oscillator basis and has a constant σ', which leads to $\mu \sim \Theta(||A||_1)$ [Fig. A.1(b)]. The one-norm of operator $(b^\dagger b)^2$ in H_2 has quadratic scaling with each transmon dimension. Since d is increased by enlarging the dimensions of all three transmons simultaneously, $||A||_1 \sim \Theta(d^{2/3})$. This scaling results in $\mu \sim \Theta(d^{2/3})$, see Fig. A.1(f).

For other Hamiltonian matrices where σ' depends on d, the scaling $\mu \sim \Theta(||A||_1)$ may still hold. In the following, I show two concrete examples—one where linearity holds, and another where it breaks. I first consider a driven system of N_q qubits with nearest-neighbor σ_z coupling.

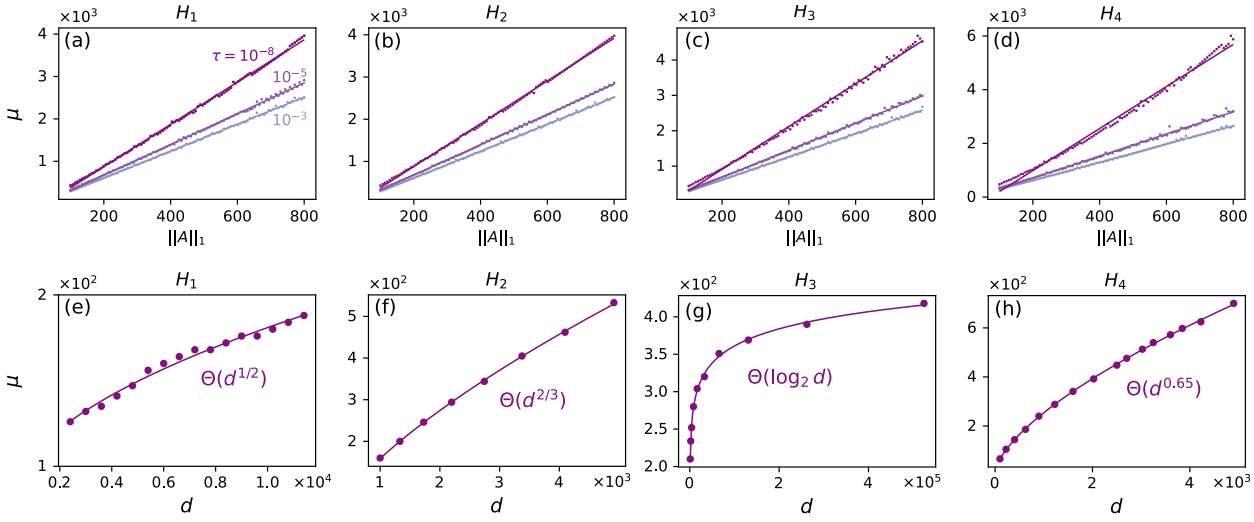

Figure A.1: Scaling of μ, the number of matrix-vector multiplications. (a)–(d) The scaling of μ with $||A||_1$ for Hamiltonians H_j, $j = 1, 2, 3, 4$, when considering several error tolerances τ. (e)–(h) The scaling of μ with dimension d when considering $\tau = 10^{-8}$ for each H_j, respectively. In all figures, dots are sampling data and solid lines are fitting curves. The parameter setup of H_1, H_2 is the same as those given in the captions of Figs. 3.1 and 3.2. Parameters of H_3 and H_4 are as follows: for H_3, $E_{Ci}/2\pi = 2.5\,\text{GHz}$, $E_{Ji}/2\pi = 8.9\,\text{GHz}$, $E_{Li}/2\pi = 0.5\,\text{GHz}$, $g/2\pi = 0.1\,\text{GHz}$, and $\phi = 0.33$; for H_4, $a_x^{(\nu)}/2\pi = a_y^{(\nu)}/2\pi = 0.5\,\text{GHz}$ and $g/2\pi = 0.1\,\text{GHz}$.

In the rotating frame, the system is described by

$$H_3(t) = \sum_{\nu=1}^{N_q} [a_x^{(\nu)}(t)\sigma_x^{(\nu)} + a_y^{(\nu)}(t)\sigma_y^{(\nu)}] + \sum_{\nu=1}^{N_q-1} g\sigma_z^{(\nu)}\sigma_z^{(\nu+1)}. \tag{A.1}$$

Each qubit is controlled via drive fields $a_x^{(\nu)}$ and $a_y^{(\nu)}$. For this example, the numerical results show that the scaling $\mu \sim \Theta(||A||_1)$ still holds, see Fig. A.1(c). Since d is increased by enlarging N_q, $||A||_1 \sim \Theta(N_q) \sim \Theta(\log_2 d)$. This scaling results in $\mu \sim \Theta(\log_2 d)$, see Fig. A.1(g). For the

second example, I consider two capacitively coupled fluxonium qubits with Hamiltonian

$$H_4(t) = \sum_{i=1}^{2} \left(4E_{Ci}n_i^2 - E_{Ji}\cos(\varphi_i) + \frac{E_{Li}}{2}[\varphi_i - \phi_i(t)]^2\right) + gn_1n_2. \tag{A.2}$$

Here, E_{Ci}, E_{Ji}, E_{Li} denote the charging, inductive, and Josephson energies of two fluxonium qubits, and g is the interaction strength. φ and n are the phase and charge number operators, defined in the usual way. The Hamiltonian is represented in the harmonic oscillator basis. Fig. A.1(d) shows that the scaling of μ is superlinear with $||A||_1$ for $\tau = 10^{-8}$. I thus settle for a numerical determination of the scaling relation between μ and d for $\tau = 10^{-8}$ by increasing the dimensional cutoff for both fluxonium qubits simultaneously. The observed scaling is $\mu \sim \Theta(d^{0.65})$, see Fig. A.1(h).

APPENDIX B

ALTERNATIVE HARD-CODED GRADIENT SCHEME: NUMERICAL IMPLEMENTATION AND SCALING

The numerical implementation of the hard-coded gradient scheme requires evaluation of the short-time propagator and its derivative. Under certain conditions, it turns out to be advantageous to numerically evaluate the short-time propagator by employing matrix diagonalization. In this appendix, I first discuss the method to compute the propagator derivative and then analyze the scaling of HG.

B.1 Evaluation of propagator derivative

The operator $-iH\Delta t$ is anti-Hermitian, and hence can be diagonalized and exponentiated according to

$$A = SDS^{\dagger}, \tag{B.1}$$

$$e^{A} = Se^{D}S^{\dagger}. \tag{B.2}$$

Here, S is unitary and D diagonal. As a result, the propagator derivative can be written as

$$\frac{de^{A}}{da} = Se^{D}\frac{dD}{da}S^{\dagger} + \frac{dS}{da}e^{D}S^{\dagger} + Se^{D}\frac{dS^{\dagger}}{da}. \tag{B.3}$$

Applying the inverse unitary transformation to $\frac{de^A}{da}$ (B.3) yields

$$S^\dagger \frac{de^A}{da} S = e^D \frac{dD}{da} + S^\dagger \frac{dS}{da} e^D + e^D \frac{dS^\dagger}{da} S. \tag{B.4}$$

Based on $\frac{dS^\dagger}{da} S + S^\dagger \frac{dS}{da} = 0$, Eq. (B.4) can be rewritten as

$$S^\dagger \frac{de^A}{da} S = e^D \frac{dD}{da} + E \circ (S^\dagger \frac{dS}{da}) \tag{B.5}$$

with $E_{ij} := e^{D_{jj}} - e^{D_{ii}}$. Here, $\circ$ denotes the Hadamard product (element-wise multiplication). To obtain dD/da and dS/da, one takes the derivative of Eq. (B.1) and repeats the same steps as for Eqs. (B.4) and (B.5), leading to

$$S^\dagger \frac{dA}{da} S = \frac{dD}{da} + E' \circ (S^\dagger \frac{dS}{da}), \tag{B.6}$$

where $E'_{ij} := D_{jj} - D_{ii}$. Since the diagonal elements of E' are zero, it follows that

$$\frac{dD}{da} = I \circ (S^\dagger \frac{dA}{da} S). \tag{B.7}$$

For off-diagonal elements of the operator in Eq. (B.6):

$$F \circ (S^\dagger \frac{dA}{da} S) + G = S^\dagger \frac{dS}{da}, \tag{B.8}$$

where

$$F_{ij} := \begin{cases} \frac{1}{D_{jj}-D_{ii}}, & \text{if } D_{jj} - D_{ii} \neq 0. \\ 0, & \text{otherwise.} \end{cases} \tag{B.9}$$

$$G_{ij} := \begin{cases} 0, & \text{if } D_{jj} - D_{ii} \neq 0. \\ (S^\dagger \frac{dS}{da})_{ij}, & \text{otherwise.} \end{cases}$$

Inserting Eqs. (B.7) and (B.8) into Eq. (B.5) gives

$$\frac{de^A}{da} = S\big((e^D + E \circ F) \circ (S^\dagger \frac{dA}{da} S)\big) S^\dagger, \tag{B.10}$$

where $E \circ G = 0$ has been used.

B.2 Scaling analysis

The gradients of the contributions to the cost function (for the case of state transfer) are evaluated by the forward-backward propagation scheme. In this scheme, the storage required for states does not scale with the number of intermediate time steps. The matrices that must be stored are the system and control Hamiltonians as well as the dense matrices S, E, and E'. Thus, the total memory usage scales as $O(d^2)$. The runtime of the scheme is dominated by $O(N)$ evaluations of the propagator and its derivative. Employing matrix diagonalization for the propagator evaluation requires a runtime scaling $\sim O(d^3)$ [146]. Calculation of the propagator derivative involves matrix-matrix multiplication and Hadamard product with runtime scaling $O(d^3)$ and $O(d^2)$, respectively. Thus, the overall runtime of the HG scheme scales as $O(Nd^3)$.

The gradients of the contributions to the gate-operation cost function are evaluated in an anal-

ogous manner, resulting in the same runtime and memory scaling.

VITA

Guy de Maupassant was born in 1850 in the Normandy region of France. He attended the University of Caen in Normandy. He volunteered for the Franco-Prussian war and then worked as a government clerk in Paris. Since middle school, he was mentored by Gustave Flaubert. At age 30, he published the “Ball of Lard”, his first major work. In the next decade, he was an extremely productive writer.